

URBANSENSE AI

Master Project & Technical Specification — v2.0

Definitive Full-System Source of Truth

AI-Powered Mobile Urban Intelligence Platform Using Public Transport Fleet

Smart India Hackathon — Problem Statement 26124 | Bharat Electronics Limited | Software | Smart Automation

Covers: team understanding · SIH PPT source · implementation source · complete PS-26124 scope · MVP · research track

1. Document Control

1.1 Purpose of v2.0

Version 1.0 of this document established the core UrbanSense architecture (mobile fleet sensing, Hardware Abstraction Layer, event-driven evidence transmission, evidence fusion, RoadTwin, lifecycle-aware road-state modeling, negative evidence, repair verification, reappearance detection, GIS, maintenance intelligence, a constrained GenAI Copilot, and a laptop-first/Jetson-later deployment path). All of that is preserved. Version 2.0 upgrades the document from an MVP-centric specification into the definitive source of truth for the complete functional scope of SIH PS 26124 — every module now shows how it scales from a first vertical slice to full PS coverage, and where genuine research innovation sits on top of that.

1.2 What changed from v1.0

- MVP is redefined as the first complete vertical slice that proves the architecture, not as a ceiling on scope (Part 72).
- A full Requirement Traceability Matrix now maps every PS-26124 capability to a module, AI method, data flow, database entity, GIS view, phase and validation metric (Part 4).
- New capability areas formalized: infrastructure expected-asset reasoning, vehicle-class breakdown, bottleneck/route-delay/OD analytics, sensor health, coverage intelligence, predictive deterioration, active sensing, and an auditable evidence-lineage layer (Parts 15–25, 41–45).
- A Full AI/CV Capability Matrix, an expanded database model, an expanded API contract, 20 GIS layers, authority workflows, 15 system-level acceptance-test scenarios, and a Definition of Done per module are added (Parts 26, 46–51, 64, 81).
- A Research/Engineering Separation Table and a Novelty Opportunity Matrix make explicit what is established technology, what is our system integration, and what remains an unvalidated research hypothesis (Parts 71/35 numbering below).
- Terminology is tightened (Part 52 vocabulary: Prototype, Edge, Simulated fleet, RoadTwin, Evidence, Observation, Event, State) and used consistently throughout.

1.3 How the team should use this document

- New teammate onboarding — read Parts 2, 6–8, then your owned parts from Part 76.
- SIH PPT — Part 83 is the slide-by-slide source; Parts 4, 26, 46, 68/72 supply the evidence.
- Implementation — Parts 8–51 give every module its interfaces; Parts 77–81 give contracts and completion criteria.
- Research — Parts 65–71 isolate the academic contribution from the engineering system.
- Demo — Part 82 is the rehearsed narrative; Part 64 is what must be proven to work.
- Testing — Parts 62–64 define what is verified before the team trusts a module or the demo.

1.4 Versioning

Version	Status	Major changes	Owner
0.x	Draft	Incremental step-by-step system design and research-strategy discussions (prior sessions)	Team (design + research logs)
1.0	Consolidated	Merged all prior sources into one 55-part Master Specification	Compiled from provided sources
2.0	This document	Upgraded to full PS-26124 scope: traceability matrix,	Compiled per upgrade brief

		capability matrices, expanded database/API/GIS, acceptance tests, Definition of Done, research/engineering separation, novelty matrix	
--	--	---	--

1.5 Status vocabulary (used consistently throughout)

Tag	Meaning
ESTABLISHED	Already decided in a prior design session; treated as a firm architectural decision.
PROPOSED	A reasonable design choice suggested here; requires team sign-off before it is treated as fixed.
IMPLEMENTED	Already built and verified working.
FUTURE	Explicitly out of scope for the SIH build; a production/roadmap item.
OPEN DECISION	Unresolved; the team must choose before implementation locks in.

1.6 Decision log (carried over + new)

Decision	Reason	Status
Develop on a laptop first; treat edge hardware as a config-only migration via a Hardware Abstraction Layer	Faster iteration for a six-person student team	ESTABLISHED
Simulate the fleet and sensors, not the city — real Indian road footage, simulated bus identity/GPS	Keeps the prototype honest without fabricating deployment claims	ESTABLISHED
Central stack: FastAPI + PostgreSQL/PostGIS + MQTT/HTTPS + object storage	Sufficient for the required workload without unnecessary infrastructure	ESTABLISHED
Road condition is a persistent, evolving state (RoadTwin), not a one-shot detection	Core differentiator over isolated CV demos	ESTABLISHED
A missed detection is evidence (subject to visibility/quality checks), not proof of absence	Enables trustworthy repair verification	ESTABLISHED
MVP means 'first vertical slice,' not 'the whole scope' — full PS-26124 coverage remains mandatory	Prevents required capabilities from being silently dropped as 'future work'	ESTABLISHED (v2.0)
Composite 'road health score' (if used) must expose its underlying components, never collapse to a single opaque number	Keeps the system auditable and prevents a meaningless AI-score presentation	ESTABLISHED (v2.0)
No innovation is added without answering: what problem it solves, why the baseline fails, why it's technically appropriate, how it's implemented, how it's evaluated, and whether it's required/research/future	Prevents buzzword-driven scope creep	ESTABLISHED (v2.0)
Research paper isolates the lifecycle-aware, uncertainty-driven, negative-evidence fusion model rather than the full platform	The full platform is engineering; the paper needs one falsifiable question	PROPOSED
Tiered fleet economics for the production pitch: majority low-cost edge nodes, a minority higher-compute survey nodes	Fiscal realism for BEL evaluators	PROPOSED

2. Project Mission

UrbanSense AI converts the existing public-transport fleet into a distributed network of mobile urban sensors, and converts their noisy, asynchronous observations into a trustworthy, continuously verified model of road, traffic, infrastructure and safety conditions across the city — closing the loop from detection through maintenance to independent, automated verification. The mission has three simultaneous audiences: our own six-member team (who must be able to build it), the SIH panel (who must see a feasible, complete answer to PS 26124), and, later, the academic reviewers of the isolated research contribution (Parts 65–71).

Guiding philosophy — AI observations are evidence, not automatically truth. Every module in this document exists to turn evidence into a state the authority can act on with confidence, and to prove — not merely assert — when that state has changed.

3. Official PS 26124 Requirements

Problem Statement 26124, 'AI-Powered Mobile Urban Intelligence Platform Using Public Transport Fleet' (Bharat Electronics Limited, Software category, Smart Automation theme), asks for an onboard-plus-centralized platform. The requirements below are restated from the official problem statement supplied as project source material, organized by capability group; Part 4 traces each one into the architecture.

3.1 Background (as given)

Buses already travel almost every major road in a city daily and increasingly carry multiple cameras (front, rear, side, cabin), but these are used only for incident recording rather than as a live sensing layer. Authorities currently depend on fixed CCTV, manual inspection, and citizen complaints to find road defects, congestion, missing infrastructure and unsafe driving — which is slow, incomplete, and hard to act on.

3.2 Required onboard (edge) capabilities

- Detect road defects: potholes, damaged roads, missing road dividers, missing zebra crossings, damaged or missing traffic signboards, waterlogging, and other road hazards.
- Estimate vehicle density through vehicle detection, classification and counting.
- Identify traffic bottlenecks.
- Detect vulnerable-pedestrian situations, including school children crossing roads.
- During incidents (hit-and-run, rash driving): detect and track the offending vehicle, extract its registration number with a confidence score, and attach a timestamp and GPS location.
- Securely share alerts with a central command system.

3.3 Required centralized-platform capabilities

- Aggregate information from the entire bus fleet.
- Visualize events on a GIS map.
- Generate congestion heat maps.
- Identify infrastructure deficiencies.
- Analyse origin–destination traffic patterns.
- Estimate route delays.
- Provide actionable insights for transport authorities.

3.4 Expected solution characteristics (as given)

- An edge-AI onboard processing framework integrated with a centralized urban intelligence platform.
- Reliable alerts, GIS-based dashboards, road-condition maps, traffic analytics and incident reports.
- Support for proactive road maintenance, improved traffic management, enhanced public safety and evidence-based decision-making.
- Bandwidth minimization through intelligent edge processing.

Scope commitment — every item in 3.2–3.4 must remain part of the final UrbanSense architecture. Difficulty is addressed by staging (MVP → Full PS → Research/Future, Part 72), never by permanent removal.

4. Requirement Traceability Matrix

Every PS-26124 requirement below is traced to: the UrbanSense module, the AI method, the edge/central split, the data flow, the database entity, the GIS view (if applicable), the implementation phase (Part 75), and the validation metric (Part 63). This is the master cross-reference — if a requirement is not in this table, it is missing from the architecture and must be added.

4.1 Road defect & infrastructure group

#	PS Requirement	Module / AI method	Edge / Central	DB entity · GIS view	Phase · Metric
1	Pothole detection	Road Defect Intelligence — detector/segmentation (Part 14)	Edge detect → Central confirm	ROAD_DEFECT · Road Health layer	Phase 4 · Precision/Recall, mAP
2	Damaged-road detection	Road Defect Intelligence — segmentation (Part 14)	Edge detect → Central confirm	ROAD_DEFECT · Road Health layer	Phase 4 · mAP, area-extent error
3	Missing/damaged road dividers	Infrastructure Intelligence — expected-vs-observed (Part 15)	Edge observe → Central compare	INFRASTRUCTURE_ASSET · Infrastructure layer	Phase 5 · Missing-asset precision
4	Missing zebra crossings	Infrastructure Intelligence — expected-vs-observed (Part 15)	Edge observe → Central compare	INFRASTRUCTURE_ASSET · Infrastructure layer	Phase 5 · Missing-asset precision
5	Damaged/missing traffic signs	Infrastructure Intelligence — detection + comparison (Part 15)	Edge detect → Central compare	INFRASTRUCTURE_ASSET · Infrastructure layer	Phase 5 · Precision/Recall
6	Waterlogging	Road Defect Intelligence — persistence-filtered detector (Part 14)	Edge detect → Central corroborate	ROAD_DEFECT · Waterlogging layer	Phase 4 · Precision/Recall
7	Other road hazards	Road Defect Intelligence — extensible hazard class (Part 14)	Edge detect → Central confirm	ROAD_DEFECT · Road Health layer	Phase 4 · Precision/Recall

4.2 Vehicle & traffic group

#	PS Requirement	Module / AI method	Edge / Central	DB entity · GIS view	Phase · Metric
8	Vehicle detection	Vehicle Intelligence — detector (Part 16)	Edge detect	VEHICLE_TRACK · Fleet/Traffic layer	Phase 3 · Precision/Recall, mAP
9	Vehicle classification	Vehicle Intelligence — classifier head (Part 16)	Edge classify	VEHICLE · Traffic layer	Phase 3 · Per-class accuracy
10	Vehicle counting	Vehicle Intelligence + Tracking (Parts 16–17)	Edge count → Central aggregate	TRAFFIC_OBSERVATION · Traffic Density layer	Phase 3 · MAE/MAPE vs. manual count
11	Traffic-density estimation	Traffic Intelligence (Part 17)	Central aggregation across passes	TRAFFIC_OBSERVATION · Congestion layer	Phase 6 · MAE/MAPE
12	Bottleneck detection	Bottleneck Detection (Part 18)	Central analytics	ROUTE_TRAVEL_TIME · Congestion/Bottleneck layer	Phase 6 · Detection precision vs. known bottlenecks
13	Route-delay estimation	Route Delay (Part 19)	Central analytics	ROUTE_TRAVEL_TIME · Route Delay layer	Phase 16 · MAE vs. scheduled time
14	Origin–destination traffic analysis	OD Analytics (Part 20)	Central analytics	ROUTE_TRAVEL_TIME · OD layer (future)	Phase 16 · Qualitative validation — see Part 20 honesty note

4.3 Safety & incident group

#	PS Requirement	Module / AI method	Edge / Central	DB entity · GIS view	Phase · Metric
15	Vulnerable-pedestrian situations	VRU Intelligence (Part 21)	Edge detect+reason	INCIDENT (risk) · Pedestrian-Risk layer	Phase 7 · Risk-alert precision
16	School-child crossing/risk	VRU Intelligence + school-zone geofence (Part 21)	Edge detect+reason	INCIDENT (risk) · School-Zone layer	Phase 7 · Risk-alert precision
17	Rash-driving detection	Incident Intelligence — trajectory anomaly (Part 23)	Edge detect	INCIDENT · Incident layer	Phase 8 · True/false alert rate
18	Hit-and-run detection	Incident Intelligence + Vehicle Tracking (Parts 24–25)	Edge detect+buffer	INCIDENT · Incident layer	Phase 8 · True/false alert rate
19	Offending-vehicle tracking	Vehicle Tracking (Part 25)	Edge track	VEHICLE_TRACK · Incident layer	Phase 8 · ID-consistency
20	Registration-number extraction (ANPR)	ANPR/OCR (Part 26)	Edge detect+OCR	PLATE · Incident (restricted) layer	Phase 9 · OCR accuracy
21	OCR/ANPR confidence, timestamp, GPS	ANPR/OCR + canonical event (Parts 26, 30)	Edge produce → Central store	EVENT, PLATE, EVIDENCE	Phase 9 · Confidence-field completeness

4.4 Platform & analytics group

#	PS Requirement	Module / AI method	Edge / Central	DB entity · GIS view	Phase · Metric
22	Secure alerting	Secure Transport + Alert Engine (Parts 31, 48)	Edge publish → Central deliver	ALERT	Phase 10 · Delivery latency/success rate
23	Centralized fleet aggregation	Central Platform + Evidence Fusion (Parts 32–33)	Central	EVENT, OBSERVATION, ROADTWIN_STATE	Phase 11–12 · Ingestion throughput

24	GIS visualization	GIS (Part 46)	Central	All GIS-layer tables	Phase 15 · Layer coverage/correctness
25	Congestion heat maps	GIS Congestion layer + Traffic Intelligence (Parts 17, 46)	Central	TRAFFIC_OBSERVATION · Congestion layer	Phase 15 · Visual/manual spot-check
26	Infrastructure-deficiency analysis	Infrastructure Intelligence + GIS (Parts 15, 46)	Central	INFRASTRUCTURE_ASSET · Infrastructure layer	Phase 5/15 · Precision/Recall
27	Actionable transport-authority insights	Maintenance Intelligence + Authority Workflows (Parts 38, 47)	Central	MAINTENANCE_TASK, ALERT	Phase 14 · Task-queue correctness
28	Edge-AI processing	Hardware Abstraction Layer + Adaptive Edge (Parts 10, 44)	Edge	EDGE_DEVICE, MODEL_VERSION	Phase 1–2 · FPS/latency benchmark
29	Bandwidth minimization	Bandwidth Strategy (Part 59)	Edge decides, Central receives	EVENT (small payload), EVIDENCE	Phase 10 · Payload size / raw-video reduction ratio

No requirement in Part 3 is left untraced: 1–7 (defects/infrastructure), 8–14 (vehicles/traffic), 15–21 (safety/incidents), 22–29 (platform/analytics) together cover every item listed in Part 3.2–3.4.

5. Problem Analysis

5.1 Current limitations

- Fixed CCTV observes only a small number of static points, missing most of the road network entirely.
- Manual inspection is infrequent, subjective, and does not scale to an entire city.
- Continuous raw-video upload is impractical — bandwidth and storage costs make city-wide always-on streaming unrealistic.
- Isolated AI detections are unreliable — a single frame's output can be a false positive/negative from glare, occlusion, or motion blur.
- Maintenance verification is difficult — authorities can mark something 'repaired' with no independent, automated confirmation.

5.2 Stakeholders

Stakeholder	Interest
Municipal/transport authority (BEL / city corporation)	Actionable, verified road and traffic intelligence at low operating cost
Bus operator / transit agency	Reuses existing fleet cameras rather than deploying new hardware everywhere
Commuters and pedestrians	Safer roads, faster incident response, fewer unaddressed hazards
SIH evaluators	A feasible, scalable, technically credible and complete answer to PS 26124
Academic reviewers (paper track)	A rigorous, falsifiable algorithmic contribution, isolated from the application layer

5.3 The actual technical problem

Design a system that converts noisy, intermittent, single-frame AI observations from a moving, uncontrolled camera platform into a trustworthy, continuously updated model of road, traffic, infrastructure and safety state — while covering every PS-26124 capability, keeping onboard compute and network bandwidth within realistic edge-device and cellular-cost budgets, and while being able to positively verify when a previously confirmed problem has been resolved.

6. UrbanSense Vision

6.1 One-paragraph explanation

UrbanSense AI turns ordinary public-transport buses into a distributed fleet of mobile urban sensors. Onboard cameras and an edge-AI pipeline detect the full PS-26124 requirement set — road defects, missing/damaged infrastructure, vehicle/traffic conditions, vulnerable-pedestrian risk and driving incidents — in real time. Instead of trusting any single detection, the system treats every observation as uncertain evidence and fuses it, over time and across multiple buses, into a persistent, evolving model of the city's roads: the RoadTwin. This lets an authority see not just 'a pothole was seen once' but 'this pothole is confirmed by three buses, has existed for eleven days, was reported for repair, and is now verified fixed.'

6.2 30-second explanation (for a judge)

City buses already drive every road, every day. We put AI on their existing cameras to spot potholes, damaged infrastructure, unsafe traffic, vulnerable pedestrians, and hit-and-run vehicles — and instead of sending raw video anywhere, we send only small, confirmed evidence packets to a central platform. Multiple buses corroborate the same spot over multiple days, so the system knows the difference between a temporary shadow and a real, persistent problem — and it can later confirm when that problem has actually been fixed.

6.3 2-minute explanation (for a mentor)

The system has two halves. On the bus (the edge), a hardware-abstracted AI pipeline processes camera, GPS and IMU streams to cover every PS-required detection task, estimates how reliable each observation is, and emits compact evidence events rather than raw video. Centrally, an evidence-fusion engine combines these asynchronous, uncertain observations — across time and across the fleet — into a per-road-segment state (RoadTwin) that moves through an explicit lifecycle from suspected to confirmed to verified-repaired, with a reappearance branch if the defect returns. This state feeds a GIS platform with authority workflows, traffic/incident/OD analytics, and a constrained GenAI Copilot that explains results but never invents them. A missing detection is treated as ambiguous, not as proof a defect is gone — this negative-evidence handling is what makes repair verification credible, and it is also the paper's core research question.

6.4 Core innovation

The differentiator is not detection — YOLO-class detectors, OCR, tracking and GIS are established technologies (Part 71). It is the lifecycle-aware, uncertainty-driven fusion of asynchronous, multi-bus observations into a persistent road state that can be corroborated, prioritized, verified as repaired, or flagged as recurring — with a non-detection itself treated as informative evidence once visibility is accounted for.

6.5 Problem → Solution mapping

See Part 4 for the complete, requirement-by-requirement mapping. Summary by capability group:

PS Requirement Group	UrbanSense Component
Road defects & infrastructure (Part 3.2)	Road Defect Intelligence (14) + Infrastructure Intelligence (15)
Vehicles & traffic (Part 3.2–3.3)	Vehicle Intelligence (16) + Traffic Intelligence (17) + Bottleneck/Route-Delay/OD (18–20)
Vulnerable pedestrians & incidents (Part 3.2)	VRU Intelligence (21) + Incident Intelligence/Rash-Driving/Hit-and-Run/Tracking/ANPR (22–26)
Secure alerting & fleet aggregation (Part 3.3)	Secure Transport (31) + Central Platform (32) + Evidence Fusion (33)
GIS, heatmaps, infrastructure deficiency, OD, route delay, actionable insight (Part 3.3)	GIS (46) + Authority Workflows (47) + Maintenance Intelligence (38)

Bandwidth minimization via edge processing (Part 3.4)	Hardware Abstraction Layer (10) + Adaptive Edge Intelligence (44) + Bandwidth Strategy (59)
---	---

7. Design Principles

7.1 Technical honesty

Clearly distinguish implemented, planned, proposed, simulated, publicly sourced, custom-collected, and future-deployment claims at all times (status vocabulary, Part 1.5). We do not currently have physical deployment on government buses, actual government-bus CCTV access, live fleet telemetry, or production bus GPS/IMU (Part 85). The prototype honestly uses real/public road footage, real Indian road geography, and simulated bus identities/fleet observations/GPS/IMU where required — simulate the fleet, not the city (Part 52).

7.2 No fake novelty

YOLO, computer vision, edge AI, bus cameras, mobile sensing, GIS, ANPR, OCR, vehicle tracking, RAG, LLMs, multi-agent systems, multi-bus observation and evidence fusion are not, individually, novel (Part 71). We distinguish existing technology, our system integration, and potential research contribution throughout.

7.3 Do not over-engineer

If a deterministic algorithm is better than an LLM, use the deterministic algorithm. If one model is enough, do not introduce several. If a fixed workflow is better than a multi-agent system, use the workflow. If PostgreSQL/PostGIS is sufficient, do not introduce an unneeded database. Every advanced component must earn its place: simple working implementation → measured baseline → advanced method → ablation → evidence of improvement — not many fashionable technologies stacked into an unfinished system (Part 47 of the upgrade brief).

7.4 SIH and research are related but different

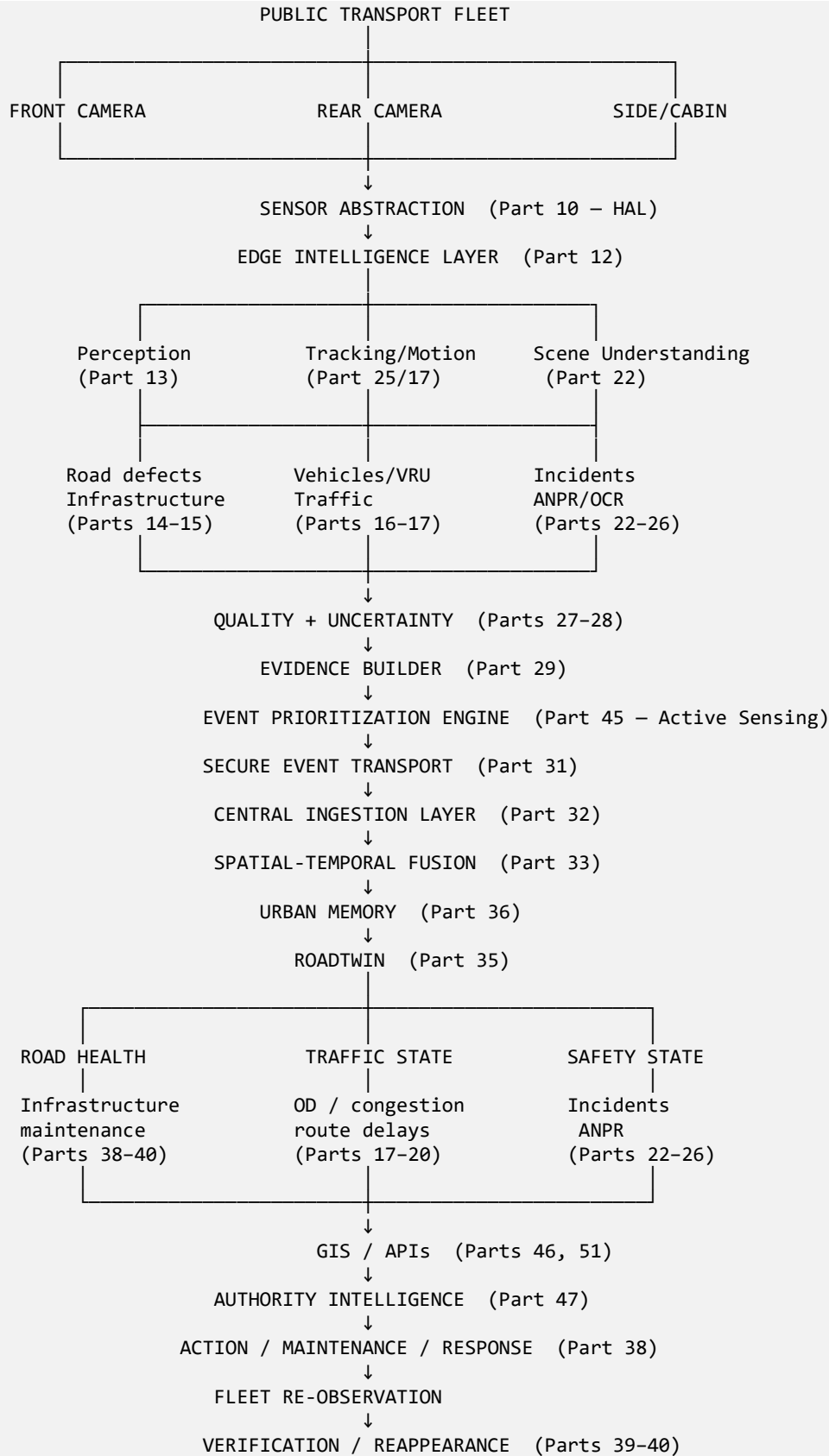
The complete UrbanSense platform (Parts 8–64) is the engineering/system solution required by PS 26124. The research paper (Parts 65–71) isolates one focused scientific question from within it. Part 35's Research/Engineering Separation Table keeps these from being confused.

7.5 Innovation admission rule

Every proposed innovation in this document (Parts 34, 38–39, 41–45, and the Innovation cross-references throughout) must answer six questions before being accepted: (1) what problem does it solve, (2) why does the baseline architecture fail without it, (3) why is it technically appropriate, (4) how would we implement it, (5) how would we evaluate it, and (6) is it required for PS coverage, a research enhancement, or future work. No multi-agent LLM swarm, blockchain, quantum computing, unnecessary transformer, unnecessary digital twin, unnecessary federated learning, unnecessary GNN, or unnecessary reinforcement learning is introduced without passing this test — and none currently does.

8. Complete System Architecture

The full architecture, expanded from the linear pipeline view into its actual branching structure:



↓
URBAN MEMORY (Part 36 – closes the loop)

8.1 Explaining every arrow

Arrow	What crosses it	Why it exists
Fleet → Sensor Abstraction	Raw camera/GPS/IMU signals from each camera role	Nothing downstream should depend on physical hardware specifics — this is the HAL boundary (Part 10).
Sensor Abstraction → Edge Intelligence	Standardized Frame/Location/IMU objects	Lets the same edge software run on laptop or Jetson-class hardware unchanged.
Edge Intelligence → {Perception, Tracking, Scene Understanding}	Scheduled frames per camera/task (Part 44 — adaptive edge)	Parallel branches because these are distinct compute workloads with different cadences (Part 13).
{Perception, Tracking, Scene Understanding} → Quality + Uncertainty	Raw model outputs (detections/tracks/classifications) plus context (angle, blur, occlusion)	No observation is trusted at face value — every output is scored before it becomes evidence (Part 27).
Quality + Uncertainty → Evidence Builder	Confidence + quality-scored candidate observations	Assembles the best supporting crop/clip and metadata for each candidate event.
Evidence Builder → Event Prioritization Engine	Candidate events with an estimated information value/urgency	Not every candidate is transmitted with the same urgency — PO safety vs. routine infrastructure differ (Part 45).
Event Prioritization → Secure Event Transport	Prioritized, compact JSON events + evidence references	Bandwidth minimization requires deciding what and when to send, not just how (Part 59).
Secure Event Transport → Central Ingestion	Authenticated MQTT/HTTPS payloads	Central platform boundary — validation and persistence happen here (Part 32).
Central Ingestion → Spatial-Temporal Fusion	Validated, deduplicated events	Raw events are not state; they must be combined across time and buses first (Part 33).
Spatial-Temporal Fusion → Urban Memory	Updated belief + full observation history	Urban Memory retains everything that contributed to a state, not just the current conclusion (Part 36).
Urban Memory → RoadTwin	The current, persisted per-segment state snapshot	RoadTwin is the canonical read layer every consumer queries (Part 35).
RoadTwin → {Road Health, Traffic State, Safety State}	Segment-level state broken out by domain	Different authority workflows care about different slices of the same segment (Part 47).
{Road Health, Traffic State, Safety State} → GIS/APIs	Queryable, visualizable state	This is the boundary between backend truth and what a human/Copilot sees (Parts 46, 51–54).
GIS/APIs → Authority Intelligence	Rendered dashboards, alerts, Copilot answers	Authorities act on interpreted state, not raw database rows.
Authority Intelligence → Action/Maintenance/Response	A decision: dispatch repair, investigate incident, adjust route	This is where the loop turns into a real-world change (Part 38).
Action → Fleet Re-observation	No direct data — a real-world change that future bus passes will encounter	The system doesn't 'know' a repair happened until it observes the result.
Fleet Re-observation → Verification/Reappearance	New observations at a previously-known segment	Closes the loop: confirms VERIFIED_REPAIRED or flags REAPPEARED (Parts 39–40).
Verification/Reappearance → Urban Memory	Updated lifecycle history	The cycle repeats — Urban Memory now contains one more complete lifecycle instance for that segment.

9. Fleet & Sensor Architecture

A bus in the UrbanSense fleet carries: a front camera, a rear camera, left/right side cameras, and an optional cabin camera (Part 11); a GNSS receiver (real in production, simulated/replayed in the prototype, Part 52); an IMU (optional); and an onboard edge-compute device (a laptop in the prototype, Jetson-class or industrial x86 hardware in production, Part 10). Each device is uniquely identified (BUS, EDGE_DEVICE, CAMERA entities, Part 50) so that every observation can be traced to its exact source (Part 25/lineage).

Sensor	Role	Prototype source	Production source
Front camera	Primary perception surface — road defects, infrastructure, oncoming traffic, pedestrians ahead	Recorded MP4 footage	RTSP/MIPI camera feed
Rear camera	Following traffic, rear hazards, hit-from-behind incidents	Recorded MP4 footage	RTSP/MIPI camera feed
Side cameras (L/R)	Side traffic, road-edge condition, flank VRU detection	Recorded MP4 footage	RTSP/MIPI camera feed
Cabin camera	Passenger-safety incidents only (Part 11, Part 43 privacy)	Recorded MP4 footage (if used)	RTSP/MIPI camera feed
GNSS	Geolocation for every observation	SimulatedGNSSProvider replaying a recorded trajectory	RealGNSSProvider
IMU	Motion context for tracking/ego-motion (Part 17)	Simulated/optional	Real IMU
Edge compute	Runs the Edge Intelligence Layer	Developer laptop (CPU/GPU)	Jetson-class or industrial x86 edge PC

10. Hardware Abstraction Layer

The edge software is built entirely behind provider interfaces so the same code runs on a laptop today and Jetson-class hardware later by swapping only the ONNX Runtime execution provider (CPUExecutionProvider → CUDAExecutionProvider → TensorrtExecutionProvider / OpenVINOExecutionProvider). ESTABLISHED.

CameraProvider

```
get_frame()
get_timestamp()
get_camera_id()
get_stream_status()
```

- FileCameraProvider — replays recorded MP4 footage (current prototype).
- RTSPCameraProvider — live camera stream (future/edge).
- USB/MIPICameraProvider — direct camera hardware (future).

LocationProvider

```
get_position()
get_timestamp()
get_accuracy()
get_heading()
```

Prototype uses a SimulatedGNSSProvider replaying a pre-recorded trajectory matching the footage; production swaps in a RealGNSSProvider.

IMUProvider

```
get_acceleration()
get_gyroscope()
get_timestamp()
```

InferenceRuntime

```
detect()
track()
segment()
ocr()
```

EventTransport

```
publish_event()
publish_health()
publish_heartbeat()
```

Why these abstractions exist: the Event Engine and AI logic must never call HTTP/MQTT, a camera device, or a GPS port directly — every dependency is injected as a provider so the agent cannot tell (and does not need to know) whether it is running on a laptop, a Jetson, or in a pure simulation.

11. Multi-Camera Architecture

Camera	Responsibilities	Frame scheduling priority	Privacy/data routing
CAM_FRONT	Road defects, infrastructure, oncoming traffic, pedestrians ahead (Parts 14–15, 17, 21)	Highest — always-on detection	Standard evidence pipeline
CAM_REAR	Rear traffic, incident/hit-and-run continuity (Parts 17, 24)	High	Standard evidence pipeline
CAM_LEFT / CAM_RIGHT	Side infrastructure, side pedestrians, road-edge condition (Parts 15, 21)	Medium	Standard evidence pipeline
CAM_CABIN	Passenger-safety incidents only	Low — event-triggered only	Privacy-restricted: excluded from road/traffic/ANPR pipelines entirely (Part 43)

Not every model runs continuously on every camera. Compute allocation (Part 44) prioritizes CAM_FRONT for always-on defect/vehicle/pedestrian detection; CAM_REAR and side cameras run at a lower duty cycle unless an event (e.g., a trajectory anomaly) raises their priority; CAM_CABIN inference is triggered only by onboard safety logic, never by road/traffic/ANPR pipelines. Camera health (frame availability, FPS, drop rate) is monitored per camera by Sensor Health (Part 41), so an absent camera feed is distinguished from a camera that legitimately saw nothing.

12. Edge AI Architecture

This section formalizes Innovation D (Adaptive Edge Intelligence) as the edge's operating model, cross-referenced in full at Part 44.

```
FAST PATH
↓ Cheap detection / scene analysis (runs on every scheduled frame)
↓
uncertainty / event trigger?
↓ yes                                ↓ no
SLOW PATH                            Continue fast-path only
↓ High-resolution model
↓ Tracking
↓ Segmentation
↓ OCR
↓ Additional evidence
```

Compute is allocated by scene complexity, uncertainty (Part 28), road-segment importance (e.g., school zone), previous observation history, safety risk, incident probability, and camera visibility (Part 45 gives the prioritization function). This reduces compute, energy, latency, and bandwidth compared to running every heavy model on every frame, while preserving detection quality on the frames that matter — the trade-off is explicit and staged (MVP heuristic vs. advanced formulation, Part 44/45), not assumed.

13. Perception System

Frame

- ↓ Preprocessing
- ↓ Frame scheduling (Part 44 – which camera/frame gets compute now)
- ↓ Detection
- ↓ Tracking
- ↓ Segmentation (where required – road surface)
- ↓ Temporal reasoning (does this persist across frames?)
- ↓ Quality estimation (Part 27)
- ↓ Event logic (does this cross a reporting threshold?)
- ↓ Evidence selection (best frame/crop to keep)
- ↓ Event creation (Part 29)

Stage	Cadence
Detection (defects, vehicles, pedestrians)	Fast-path, always-on at a sampled frame rate
Tracking	Fast/slow-path, while relevant objects are in frame
Segmentation	Slow-path — periodic/triggered
OCR	Slow-path — event-triggered only (incident-driven)
Health/heartbeat	Periodic, low-frequency batch (Part 41)

14. Road Defect Intelligence

Covers PS requirements 1, 2, 6, 7 (Part 4.1).

14.1 Potholes

Aspect	Detail
Input	CAM_FRONT frames
Model	Object detector / segmentation model (variant OPEN DECISION, Part 56/86)
Algorithm	Detect → track across frames → confirm persistence before treating as a candidate
Post-processing	Bounding-box/mask smoothing across frames from the same pass
Geolocation	GPS + map-matched road-segment ID
Confidence	Model output probability
Quality	Visibility/motion-blur/angle score (Part 27)
Evidence	Best-quality crop + confidence + quality retained as evidence
Central fusion	Corroborated across buses/passes before CONFIRMED (Part 33)
RoadTwin state	OBSERVED → CANDIDATE → CONFIRMED (Part 37)
Authority output	Prioritized maintenance entry with location, severity, recurrence

14.2 Damaged roads

Aspect	Detail
Input	CAM_FRONT frames
Model	Segmentation model (surface-damage class)
Algorithm	Area-based severity estimate from mask extent
Post-processing	Merge adjacent damaged segments into one defect region
Geolocation	GPS + segment ID
Confidence	Model probability
Quality	Same quality model as 14.1
Evidence	Damage-extent evidence + crop
Central fusion	Same corroboration logic as potholes
RoadTwin state	Same lifecycle as potholes
Authority output	Same as 14.1, area/extent included

14.3 Waterlogging

Aspect	Detail
Input	CAM_FRONT frames, ideally during/after rain
Model	Detector/segmentation trained on standing-water appearance + custom data
Algorithm	Persistence check (transient puddle vs. standing waterlogging)
Post-processing	Temporal filter — must persist across a short window to avoid false positives from wet-road glare
Geolocation	GPS + segment ID
Confidence	Model probability
Quality	Illumination/reflection strongly affects this — quality score weights glare
Evidence	Crop + timestamp (time-of-day/weather correlation informative)
Central fusion	Cross-bus corroboration especially valuable (rain event ≠ persistent waterlogging)
RoadTwin state	Same lifecycle
Authority output	Flagged separately from structural defects since cause differs

14.4 Other hazards (general)

Aspect	Detail
Input	CAM_FRONT/side cameras
Model	Extensible detector for ad-hoc hazard classes (debris, fallen objects, etc.)
Algorithm	Same detect → track → persist pattern
Post-processing	Same as above
Geolocation	GPS + segment ID
Confidence	Model probability

Quality	Same quality model
Evidence	Crop + confidence
Central fusion	Same corroboration pattern
RoadTwin state	Same lifecycle
Authority output	General hazard alert to authority

15. Infrastructure Intelligence

Covers PS requirements 3, 4, 5, 26 (Part 4.1, 4.4). Formalizes Innovation J — missing infrastructure is never forced into ordinary object detection, since you cannot 'detect the absence of a class' with a bounding-box model alone; it requires a reference to compare against.

```

Expected asset (from GIS inventory): e.g., sign at GPS = X
      ↓
Bus observations near X across multiple passes
      ↓
Was the expected location actually observable this pass? (Part 34 – negative-evidence gate)
      ↓ yes                                ↓ no
Present / Damaged / Absent / Uncertain    Pass discarded – inconclusive

```

15.1 Traffic signs — damaged or missing

Aspect	Detail
Input	CAM_FRONT frames + GIS expected-sign reference
Model	Detector for sign presence/condition + expected-vs-observed comparison
Algorithm	'Missing' requires an expected-location reference; 'damaged' is a direct detection
Post-processing	Cross-check against GIS reference layer before declaring 'missing'
Geolocation	GPS + segment ID + expected-location match
Confidence	Detector probability for damage; comparison confidence for missing
Quality	Visibility (occlusion by trees/vehicles) is critical here
Evidence	Crop of sign (or of the empty expected location) as evidence
Central fusion	Multiple passes required before 'missing' is confirmed (single-pass occlusion is common)
RoadTwin state	Same lifecycle
Authority output	Separate 'damaged' vs 'missing' authority categories

15.2 Missing dividers / missing zebra crossings

Aspect	Detail
Input	CAM_FRONT frames + GIS expected-infrastructure reference
Model	Expected-vs-observed comparison (same pattern as 15.1)
Algorithm	Never treated as plain object detection — always a comparison against a reference
Post-processing	Confirm across multiple passes and lighting conditions
Geolocation	GPS + segment ID
Confidence	Comparison confidence
Quality	Lane-marking visibility (paint wear, glare, rain) is the dominant quality factor
Evidence	Crop + reference-layer match result
Central fusion	Requires strong corroboration before confirming absence — high false-positive risk from occlusion
RoadTwin state	Same lifecycle
Authority output	Explicit 'missing infrastructure' authority category

15.3 Infrastructure inventory

Maintained as a GIS-referenced `INFRASTRUCTURE_ASSET` table (Part 50) — each expected asset has a type, an expected location, and a rolling observation history. This inventory is what makes 'missing' a meaningful, checkable claim rather than a guess; without it the system could only ever report what it happened to see, never what it expected to see and didn't.

16. Vehicle Intelligence

Covers PS requirements 8, 9 (Part 4.2). This section is the vehicle-class slice of the Full AI/CV Capability Matrix (Part 26).

Vehicle class	Detection note	Indian-context relevance
Car	Standard detector class	Baseline class, well covered by public datasets
Bus	Standard detector class — also used to identify fleet-relevant vehicles in scene	Well covered
Truck	Standard detector class	Well covered
Motorcycle / two-wheeler	Requires higher-density handling — far more common on Indian roads than in most public datasets	High-priority fine-tuning target (Part 56)
Auto-rickshaw	Not a standard class in most Western datasets	High-priority fine-tuning target (Part 56); requires custom-labeled data
Bicycle	Standard-ish class, but interaction logic (VRU-adjacent) matters more than raw detection	Overlaps with VRU risk reasoning (Part 21)

Classification feeds directly into counting (density estimation) and into the traffic/VRU modules; a pedestrian-adjacent class (bicycle) is deliberately flagged for VRU interaction logic (Part 21) rather than being treated purely as 'traffic.'

17. Traffic Intelligence

Covers PS requirements 10, 11, 25 (Part 4.2, 4.4). Formalizes Innovation K — the camera is moving, so traffic estimation cannot use a fixed pixel ROI the way static CCTV analytics do.

Vehicle Detection (Part 16) → Classification → Tracking (Part 25)

↓

Ego-motion correction (using GPS/heading/speed from the HAL, Part 10)

↓

Dynamic Spatial Reasoning (relative displacement + ego-motion, not a fixed ROI)

↓

Counting → Density → Flow → Speed → Congestion → Bottleneck (Part 18) → Route Delay (Part 19)

Per-segment vehicle counts and speeds from many bus passes are aggregated centrally into density, flow, congestion and bottleneck estimates, and combined with route/GPS history for origin–destination pattern and route-delay analytics (Parts 19–20).

18. Bottleneck Detection

Covers PS requirement 12. A bottleneck is inferred centrally, not on a single bus: repeated, corroborated low-speed/high-density readings at the same segment across multiple passes and multiple buses, distinguished from a one-off slowdown (e.g., a single stopped vehicle) by persistence over a time window. Output is a bottleneck flag on the segment's traffic state (RoadTwin, Part 35), feeding the Congestion/Bottleneck GIS layer (Part 46) and route-delay estimation (Part 19).

19. Route Delay

Covers PS requirement 13. Formalizes part of Innovation L. Route delay is estimated by comparing observed segment/route travel time (derived from consecutive GPS-timestamped positions of buses on a known route) against an expected/baseline travel time for that route or time-of-day. This requires only the fleet's own trajectory data — it does not require external traffic-data feeds — but its accuracy is bounded by how many buses traverse a given route and how often (an OPEN DECISION-sensitive metric, Part 63).

20. OD Analytics

Covers PS requirement 14. Formalizes the remainder of Innovation L. Fleet movement observations can approximate: route travel time, segment travel time, recurring delay, bottleneck locations, and temporal congestion patterns with reasonable confidence, because these derive directly from the fleet's own known routes and timestamps.

Honesty constraint — a true city-wide origin–destination matrix requires passenger-level trip data (where individual travelers start and end), which a bus-mounted sensing fleet does not observe. UrbanSense can approximate route-level and corridor-level flow patterns from fleet trajectories; it must not claim a full OD matrix without additional data sources. This is FUTURE / requires validation, not an established capability.

21. Vulnerable Road User (VRU) Intelligence

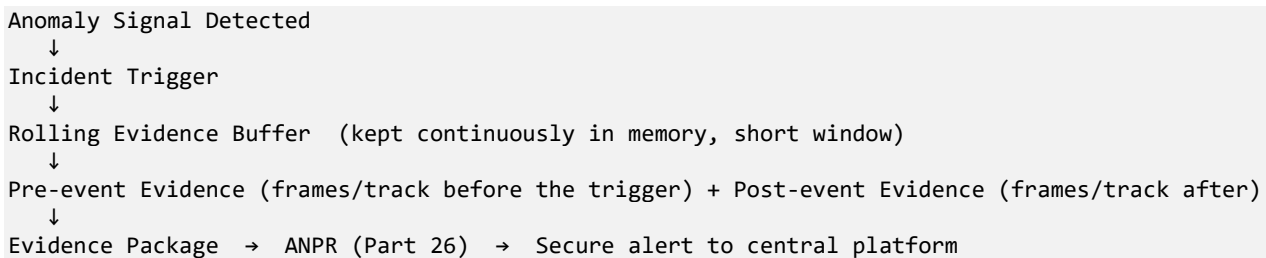
Covers PS requirements 15, 16 (Part 4.3).

- Pedestrian detection — CAM_FRONT/side detector output.
- Tracking — multi-object tracking to establish pedestrian trajectory, not just presence.
- School-zone geofencing — GIS reference layer marks zones needing elevated sensitivity.
- Vehicle–pedestrian interaction — combine pedestrian and vehicle trajectories to detect closing distance/conflict, not just co-occurrence.
- Trajectory prediction — short-horizon extrapolation to flag an emerging unsafe interaction before it happens.
- Risk scoring — combines proximity, closing speed, zone sensitivity (e.g., school zone) and visibility.
- Alert generation — only unsafe-interaction events are surfaced as alerts; plain pedestrian detections are analytics, not alerts.

Key distinction — person detection (a CV output) is not the same thing as unsafe-interaction detection (a reasoning output over trajectories + context). The system must not conflate the two when generating alerts.

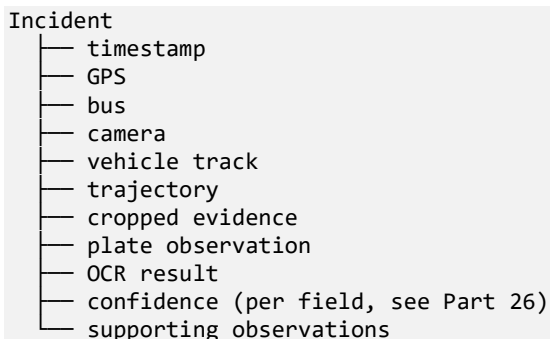
22. Incident Intelligence

Covers general incident handling underlying PS requirements 17, 18 (Part 4.3).



The rolling buffer is what makes 'pre-event' evidence possible: the edge agent must always keep a short recent window of frames/track history, because the trigger condition is only recognizable after the fact. Every incident is structured as an evidence graph (Part 22.1) so the authority can trace exactly why an alert fired.

22.1 Incident evidence graph



This is the auditable evidence chain formalizing Innovation M — an authority reviewing an incident alert can trace it back to the original frames/observations that produced it, not just a summary claim (Part 25 — Digital Evidence Lineage — applies this pattern to RoadTwin state as well).

23. Rash Driving

Covers PS requirement 17. Detected via trajectory/kinematic anomaly detection: sudden acceleration or deceleration, erratic lane change, abnormal closing speed relative to surrounding vehicles, or an abnormal

heading change rate, computed over the tracked vehicle's trajectory (Part 25) combined with the bus's own ego-motion (Part 17). Triggers the Incident Intelligence rolling-buffer workflow (Part 22).

24. Hit-and-Run

Covers PS requirement 18. Detected as an anomalous contact/near-contact event (a sudden trajectory discontinuity consistent with a collision, or a stationary-then-fleeing pattern) followed by the offending vehicle leaving the scene. Triggers Incident Intelligence (Part 22), Vehicle Tracking (Part 25) to keep following the offending vehicle as long as possible, and ANPR (Part 26) to extract its registration.

24.1 Cross-camera / cross-bus vehicle continuity

Formalizes Innovation N. An offending vehicle may leave CAM_FRONT's view and enter CAM_REAR's, or leave one bus's sensing range and later be observed by another bus.

```
CAM_FRONT track
  ↓ (vehicle passes out of front view)
CAM_REAR track (same bus)
  ↓ (vehicle continues down the route)
Another bus's observation (different bus, later time)
```

Continuity across these hops is estimated probabilistically using appearance features (color/type/class), trajectory consistency, timestamp plausibility, GPS/direction consistency, and — when available — plate evidence. This is designed as probabilistic vehicle continuity, not claimed re-identification: the system reports a continuity confidence, never an assumed-certain identity match. Cross-bus continuity beyond simple appearance+trajectory heuristics is a Research-tier extension (Part 71), not required for the MVP or Full PS baseline (Part 72–73).

25. Vehicle Tracking

Multi-object tracking (MOT) maintains consistent object identity for vehicles across frames within a camera's view, feeding rash-driving/hit-and-run reasoning (Parts 23–24), traffic density/flow (Part 17), and cross-camera continuity (Part 24.1). Tracking quality (ID-switch rate) is a first-class evaluation metric (Part 63), since a broken track can silently corrupt both incident evidence and traffic counts.

26. ANPR/OCR

Covers PS requirements 19, 20 (Part 4.3).

```
Incident Trigger
  ↓
Vehicle Localization
  ↓
Plate Detection
  ↓
Multi-frame Cropping (best of several frames, not a single crop)
  ↓
OCR
  ↓
Candidate Fusion (combine multi-frame OCR reads)
  ↓
OCR Confidence
  ↓
Evidence Package
```

Three distinct confidence values must be tracked and reported separately, never collapsed into one number:

- Vehicle detection confidence — is this a vehicle, and is it the one implicated in the incident?
- Plate detection confidence — was a plate region actually localized?

- OCR confidence — how reliable is the extracted text itself?

Privacy/security: plate text and crops are handled as sensitive evidence, restricted to authorized incident-review roles and retained only per the retention policy (Part 49), never exposed as a general-purpose surveillance/search capability (Part 49.1).

27. Observation Quality

Potential quality factors: image visibility, motion blur, camera angle, illumination, occlusion, GPS accuracy, tracking stability, and observation duration.

Core principle — Confidence ≠ Quality ≠ Severity ≠ Priority. These are four different axes and must never be collapsed into one score.

Axis	Question it answers	Example
Confidence	How sure is the model that what it detected is real?	0.92 probability this is a pothole
Quality	How trustworthy is the observation itself, independent of the model's confidence?	Clear daylight, stable frame, good angle → high quality; blurry night frame → low quality even if confidence is high
Severity	How bad is the underlying condition?	A hairline crack vs. a deep pothole spanning the lane
Priority	Given everything (severity, traffic, recurrence, quality of evidence), how urgently should this be acted on?	A moderate pothole on a high-traffic school route may outrank a severe pothole on an unused lane

Example: a high-confidence (0.95) detection from a badly-angled, motion-blurred frame is high-confidence but low-quality — it should not by itself confirm a defect. A low-confidence (0.4) detection corroborated by three independent, high-quality observations from different buses may still justify confirmation.

28. Uncertainty

Formalizes Innovation O. Four distinct notions are kept separate rather than collapsed into one number:

Concept	Meaning
Model confidence	'The detector thinks this is a pothole.'
Observation quality	'The image was clear enough to trust' (Part 27).
Epistemic uncertainty	'The model has limited knowledge of this situation' — reducible with more/better data.
Aleatoric uncertainty	'The environment itself is ambiguous' — irreducible sensor/scene noise (rain, glare).

Uncertainty routes an observation into one of four actions rather than forcing a binary decision:

Confirm
↓ (if evidence insufficient)
Keep as candidate
↓ (if still ambiguous)
Request stronger evidence (raise priority for future high-fidelity passes, Part 45)
↓ (if evidence remains inconclusive after reasonable effort)
Abstain (explicitly report 'insufficient evidence', not a guess)

Tier	What it includes	Status
MVP	Raw model confidence + the quality score (Part 27), combined with a straightforward multi-observation corroboration rule	IMPLEMENTED-target for MVP
Advanced	Explicit aleatoric/epistemic decomposition; calibrated confidence via temperature scaling or similar	PROPOSED — after MVP is stable
Research	Evidential deep learning (Dirichlet output heads) and/or conformal prediction with coverage guarantees for the detector and for road-state estimation	Research track (Part 69); not required for the SIH demo

29. Evidence Builder

Sits between raw perception output and the transportable Event Contract (Part 30). For each candidate observation that crosses a reporting threshold, the Evidence Builder: selects the best-quality supporting frame/crop (or a short clip for incidents, Part 22), attaches the confidence and quality scores (Parts 27–28), attaches GPS/map-matched segment ID, attaches the model/version identifier (for lineage, Part 25 upgrade-brief concept — formalized centrally in Part 33.1), and hands the assembled evidence package to the Event Prioritization Engine (Part 45) before transmission.

30. Event Contract

Baseline canonical event schema (expand fields per module during implementation; this is the shared contract every team member codes against):

```
{
  "event_id": "uuid",
  "bus_id": "string",
  "camera_id": "string",
  "timestamp": "ISO-8601",
  "location": {
    "lat": 0.0,
    "lon": 0.0,
    "road_segment_id": "string"
  },
  "event_type": "pothole | damaged_road | missing_sign | missing_divider |
    missing_zebra_crossing | waterlogging | hazard |
    vehicle_count | pedestrian_risk | incident | anpr |
    traffic_observation | sensor_health | ...",
  "confidence": 0.0,
  "quality": 0.0,
  "model_version": "string",
  "evidence_ref": "object-storage-key-or-url",
  "schema_version": "1.0"
}
```

Field	Required?	Type	Notes
event_id	Required	UUID	Globally unique per event
bus_id	Required	string	Simulated in prototype (Part 52)
camera_id	Required	string	One of the roles in Part 11
timestamp	Required	ISO-8601	Edge-device clock, UTC recommended
location.lat/lon	Required	float	From LocationProvider (Part 10)
location.road_segment_id	Required	string	Map-matched at edge or centrally — OPEN DECISION
event_type	Required	enum/string	Extensible; new types added by module owners via change log (Part 1.6)
confidence	Required	float 0–1	Model confidence — see Part 27 for why this is not sufficient alone
quality	Required	float 0–1	Observation-quality score — Part 27
model_version	Required	string	Enables the evidence-lineage requirement (Part 33.1)
evidence_ref	Optional	string	Omitted for pure telemetry/heartbeat/sensor-health events
schema_version	Required	string	Enables backwards-compatible evolution

Versioning/compatibility: new optional fields may be added without a version bump; required-field changes or semantic changes to an existing field require a schema_version increment and a migration note in the decision log.

31. Secure Transport

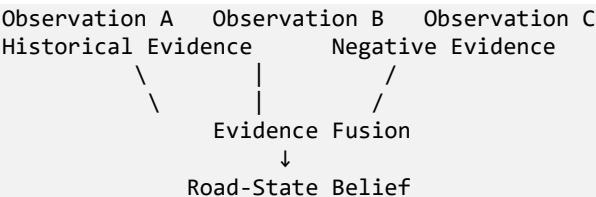
Dual-priority routing (PROPOSED refinement, carried from v1.0): real-time P0/P1 safety events (incidents, ANPR) go out immediately over cellular MQTT/HTTPS as small payloads; bulkier P2/P3 routine infrastructure logs (pothole/waterlogging/sign observations) can be batched and synced opportunistically (e.g., over depot Wi-Fi) to minimize cellular cost — see Part 59 for the full bandwidth strategy. All transport is authenticated and encrypted (TLS for HTTPS, TLS-secured MQTT), per Part 49.

32. Central Platform

- API Gateway / FastAPI — all ingestion and query endpoints (Part 51).
- Event ingestion — validates and persists incoming events; publishes onward to the fusion engine.
- Deduplication — collapses retried or multi-camera-overlap reports of the same physical event before they are double-counted (Part 33.1, Innovation C).
- Spatial map matching — resolves GPS to a canonical road-segment ID, correcting for GPS drift.
- Evidence storage — object storage for crops/clips referenced by event records.
- PostgreSQL — primary relational store (Part 50).
- PostGIS — geospatial extension for road segments, spatial joins, map matching.
- Redis (optional) — caching / low-latency lookups if needed at scale.
- Fusion engine — implements Part 33's evidence-fusion logic.
- RoadTwin / Urban Memory — persisted state and history layer (Parts 35–36).
- Traffic / OD / route-delay / infrastructure / incident analytics services (Parts 17–20, 15, 22).
- Maintenance intelligence, alert engine, GIS, fleet monitoring, Copilot, audit trail, monitoring (Parts 38, 48, 46, 41–42, 52, 61).

Edge Event → MQTT/HTTPS → FastAPI Ingestion → Validation → Deduplication
→ Evidence Store (object storage) + Event Table (PostgreSQL)
→ Map Matching → Fusion Engine → RoadTwin (PostGIS) → Analytics
→ GIS / API / Copilot → Alert Engine → Authority

33. Evidence Fusion



Fusion considers: spatial agreement (same segment/location), temporal agreement (same time window or consistent across time), source reliability (device/camera history), model confidence, observation quality, historical consistency, camera visibility at the time, and negative evidence (Part 34). Multiple buses passing the same segment and independently reporting the same condition corroborate each other far more strongly than repeated detections from a single bus/camera, which may share a systematic bias (e.g., a persistently misaligned camera).

33.1 Cooperative mobile sensing — independence vs. duplication

Formalizes Innovation C. Multiple buses are treated as asynchronous cooperating observers, but the fusion engine must distinguish four cases before counting corroboration:

Case	Description	Counted as corroboration?
Independent observations	Different buses, different passes, no shared camera/frame	Yes — this is genuine corroboration
Duplicate observations	The same physical detection reported twice (e.g., a retried event, or two cameras on the same bus catching the same object in the same pass)	No — deduplicated before fusion (Part 32)
Correlated observations	Different buses, but sharing a common failure mode (e.g., all passing during the same rainstorm, or all using the same faulty model version)	Down-weighted — treated as partially, not fully, independent
Genuinely independent corroboration	Different buses, different times, different conditions, consistent detection	Strongest possible evidence for CONFIRMED

Example: BUS-001 reports a pothole at confidence 0.82, BUS-014 at 0.91, BUS-027 reports no detection due to poor visibility (excluded per Part 34's quality gate, not counted as negative evidence), and BUS-032 reports at 0.87. Three genuinely independent positive observations, with the fourth pass correctly excluded rather than miscounted as an 'absence,' is what justifies CONFIRMED — not the raw count of buses that happened to pass.

34. Negative Evidence

Formalizes Innovation B. A non-detection must NOT automatically be read as absence.

Did the camera see the relevant area?

↓ yes

Was visibility adequate?

↓ yes

Was occlusion acceptable?

↓ yes

Was motion blur acceptable?

↓ yes

Was GPS/map matching reliable?

↓ yes

Was the expected asset within the camera's observable region?

↓ yes

CONFIDENT ABSENCE

↓ (any 'no' at any gate)

INCONCLUSIVE PASS — discarded, not counted as evidence of absence

Only once accumulated evidence quality is sufficient does the system declare Confident Absence — this staged gate (rather than a single blanket 'no detection = OK' assumption) is what makes repair verification (Part 39), missing-infrastructure detection (Part 15), and reappearance detection (Part 40) trustworthy rather than a guess. This is a concrete research opportunity (Part 68 ablation: 'no negative evidence') — quantifying exactly how much each gate contributes to correct absence declarations is an open empirical question, not asserted here.

35. RoadTwin

RoadTwin is the canonical, per-segment urban-state layer — the persistent 'digital twin' of road condition, not a 3D city model. Possible state fields per road segment:

- Road condition (defect-free / degraded / severely degraded).
- Defect state (list of active/CONFIRMED defects with lifecycle status, Part 37).
- Infrastructure state (signs, dividers, crossings — present/missing/damaged, Part 15).
- Traffic state (current density/flow/congestion class, Parts 17–19).
- Incident state (recent incidents at/near the segment, Part 22).
- Maintenance state (queued/in-progress/resolved, Part 38).
- Uncertainty (confidence in the current state, Part 28).
- Historical observations (link into Urban Memory, Part 36).
- Last observation (timestamp + contributing bus).
- Recurrence (has this state repeated after resolution?, Part 40).
- Coverage/freshness metadata (Parts 42, 24 upgrade-brief concept — see 35.2).

RoadTwin is the canonical state layer because every downstream consumer — GIS, maintenance prioritization, the Copilot, analytics, authority workflows — reads from it rather than recomputing state from raw events; this keeps 'what is currently true about this road' defined in exactly one place.

35.1 Road Segment Health Score

Formalizes Innovation F. RoadTwin exposes each dimension of a segment's condition separately — it does not collapse them into one opaque 'AI score.'

```
Road Health
├── Surface condition
├── Pothole probability
├── Waterlogging probability
├── Marking condition
├── Divider condition
├── Sign condition
├── Traffic condition
├── Incident risk
├── Pedestrian risk
├── Maintenance status
├── Observation confidence
└── Historical degradation trend
```

Rule — if a single composite score is displayed for at-a-glance scanning (e.g., in a summary dashboard tile), its exact mathematical basis (which components, what weights) must be shown alongside it, and the full component breakdown must always remain one click away. A composite score is a view, never the stored representation — the underlying components in the tree above are what is actually persisted (Part 50).

35.2 Observation freshness

Formalizes Innovation S. Road state is not trusted equally forever.

```
Fresh observation
↓
High confidence in current state

Older observation, no recent re-check
↓
Confidence decay over time
```

No recent observation at all
↓
State marked STALE – flagged distinctly from a confidently-current state

New observation arrives
↓
State refreshed, confidence restored per Part 33

Exact decay function is an OPEN DECISION (Part 86) — the requirement is that the GIS/API/Copilot must be able to distinguish a confidently-current CONFIRMED state from a CONFIRMED state that hasn't been re-checked in a long time, since the two carry different operational meaning.

36. Urban Memory

A road segment retains a persistent historical record, not just current status:

- Observations (all raw evidence contributing to its state over time).
- Defects (current and historical, including resolved ones).
- Traffic history (density/flow trends).
- Incidents recorded at or near the segment.
- Maintenance actions taken.
- Repairs and their verification outcomes.
- Recurrence history (has this segment had repeat problems?).
- Uncertainty history (how confidence evolved over time).

This matters because a segment with three past repairs of the same defect type is a meaningfully different maintenance-planning case than a segment seeing its first-ever defect — even if their current CONFIRMED state looks identical. Urban Memory is also the substrate for Predictive Road Intelligence (Part 43) and for the digital evidence lineage requirement (Part 33.1 concept, formalized centrally as an audit capability, Part 42).

37. Road Lifecycle

Established lifecycle for a road defect:

OBSERVED
↓ validation
CANDIDATE
↓ corroboration / persistence across buses or passes
CONFIRMED
↓ priority + authority action
MAINTENANCE_PENDING
↓ authority reports action taken
REPAIR_REPORTED
↓ awaiting future fleet passes
VERIFICATION_PENDING
↓ sustained confident absence (Part 34)
VERIFIED_REPAIRED
↓ new positive evidence at the same location
REAPPEARED (recurring defect, Part 40)

Transition	Trigger	Required evidence	Confidence/quality gate	DB update	Authority action
OBSERVED → CANDIDATE	First detection passes basic validation	Single observation above a minimum confidence	Model confidence only	Create observation + candidate record	None yet
CANDIDATE → CONFIRMED	Corroboration/persistence threshold met	≥ N independent high-quality	Confidence AND quality both above threshold	Update defect status = CONFIRMED; link	Enters maintenance priority queue

		observations (N: OPEN DECISION)		contributing observations	
CONFIRMED → MAINTENANCE_PENDING	Priority computation + authority acknowledgement	Priority score (Part 38)	N/A (deterministic scoring, not model confidence)	Set priority + queue position	Assigned to maintenance workflow
MAINTENANCE_PENDING → REPAIR_REPORTED	Authority marks repair action taken	Manual authority input	N/A	Timestamp repair report	Await verification
REPAIR_REPORTED → VERIFICATION_PENDING	Automatic on repair report	N/A	N/A	Flag segment for verification priority (Part 45 — raises re-observation priority)	None — passive/prioritized monitoring
VERIFICATION_PENDING → VERIFIED_REPAIRED	Sustained confident absence across future passes	Multiple high-quality non-detections (Part 34)	Absence-confidence threshold (OPEN DECISION)	Set status = VERIFIED_REPAIRED; close maintenance record	Notify authority; repair confirmed
VERIFIED_REPAIRED → REAPPEARED	New positive detection at same segment	New CANDIDATE-level evidence at a previously-repaired location	Same as OBSERVED→CANDIDATE	Link new observation to historical defect record (Part 36)	Flagged as recurring — may raise priority (Part 38)

38. Maintenance Intelligence

Formalizes Innovation H. Priority is kept explicitly separate from AI confidence and from raw defect severity:

Concept	Meaning
AI confidence	How sure the model is that the defect exists (an input, not the final answer).
Defect severity	How bad the physical condition is (extent/depth/type).
Operational priority	Where this sits in the authority's action queue, combining severity with operational context.

$\text{Priority} \propto \text{severity} \times \text{traffic_exposure} \times \text{pedestrian_vulnerability} \times \text{school_proximity} \times \text{persistence} \times \text{confidence} \times \text{road_importance}$

MVP heuristic — a simple weighted sum or ranked-tier rule using the factors above, with weights PROPOSED and explicitly not validated.

Configurable authority policy — weights exposed as a configuration the authority can tune per their own operational priorities, rather than hard-coded.

Research possibility — learned prioritization (e.g., ranking-model trained on historical authority action data) is a Research-tier extension (Part 71) — not attempted without real historical prioritization data, which we do not currently have.

- Severity
- Traffic volume on the segment
- Recurrence (has this happened before here?)
- Persistence (how long has it been CONFIRMED?)
- School-zone / high-risk proximity
- Route importance (arterial vs. minor road)
- Number of independent corroborating buses
- Observation quality of the contributing evidence

39. Repair Verification

Formalizes Innovation I — the closed-loop cycle that is a primary UrbanSense differentiator over a simple 'detect → alert' pothole spotter.

```
Detect → Confirm → Prioritize → Maintenance → Repair
    → Re-observe → Verify → Monitor → Detect recurrence
Repair Reported
↓
Verification Pending
↓
Future Fleet Passes (any bus passing the same segment)
↓
Observation Opportunity (was the segment actually visible this pass? – Part 34)
↓
Positive Evidence (defect still seen) / Negative Evidence (not seen, Part 34)
↓
Belief Update
↓
Verified Repaired (only after sufficient confident-absence evidence accumulates)
```

Why this is more valuable than a simple pothole detector

A one-shot detector answers 'is there a pothole here right now, in this frame?' It cannot tell an authority whether a reported repair actually happened, whether a defect keeps coming back, or how confident to be in either answer. The closed loop turns detection into accountable, auditable maintenance history.

Failure cases to design for

- A bus passes but the segment is occluded (traffic/parking) — must not count as verification evidence (Part 34 gate).
- A partial repair (defect reduced but not eliminated) — should not falsely trigger VERIFIED_REPAIRED from a single ambiguous pass.
- Authority reports a repair that was not actually done — the system should surface persistent positive evidence rather than silently trusting the manual report.

40. Reappearance Detection

```
VERIFIED_REPAIRED
↓ new evidence at same segment
REAPPEARED
```

The value to authorities: distinguishing a brand-new defect from a recurring one at the same location changes the maintenance response — a recurring defect may indicate an underlying structural issue (drainage, base-layer failure) rather than a simple resurfacing need, and repeated recurrence at the same segment is itself a useful signal for prioritizing a deeper fix rather than another patch. Recurrence count and history are stored in Urban Memory (Part 36) and are an explicit input to Predictive Road Intelligence (Part 43).

41. Sensor Health

Formalizes Innovation Q — self-diagnosing edge AI. The edge device monitors its own operating condition, not just the road:

- Camera availability (per camera role, Part 11).
- FPS actually achieved vs. target.
- Inference latency per stage (fast-path/slow-path, Part 12).
- Dropped frames.
- GPU/CPU utilization, memory.
- Temperature (where available on the target hardware).
- GPS quality (accuracy field from the LocationProvider, Part 10).
- Network status.
- Queue depth (Part 59).
- Model failures/exceptions.

Why this matters — the platform must be able to distinguish 'no potholes detected' from 'the camera was not functioning, so absence cannot be trusted.' Without sensor-health awareness, a dead camera would silently masquerade as a clean road — which would directly corrupt Negative Evidence (Part 34) and Repair Verification (Part 39). Sensor-health events are a first-class event_type in the canonical schema (Part 30) and feed the Coverage Intelligence layer (Part 42).

42. Coverage Intelligence

Formalizes Innovation R. Built from Sensor Health (Part 41) and observation history (Urban Memory, Part 36), a fleet-wide coverage map distinguishes:

- Strong observation coverage — frequent, high-quality passes.
- Weak coverage — infrequent or low-quality passes.
- No recent observation — a segment the fleet hasn't covered in a meaningful window.
- Poor camera visibility — passes occurred but under conditions unlikely to have produced trustworthy evidence.
- Insufficient corroboration — observed, but not by enough independent sources to reach CONFIRMED (Part 33.1).

This produces an Urban Sensing Coverage Map, exposed as its own GIS layer (Part 46), so authorities can see not only 'what do we know?' but also 'where are we uncertain because we haven't observed enough?' — a distinction a plain detection dashboard cannot make. This is a genuinely useful extension of mobile sensing over fixed CCTV, since fixed cameras have a coverage question too, but bus routes make coverage dynamic and improvable (e.g., by adjusting route emphasis), which is itself a FUTURE planning input, not implemented in the SIH build.

43. Predictive Road Intelligence

Formalizes Innovation G — clearly labeled as predictive/research intelligence, not a claim that the necessary real-world maintenance/weather data already exists.

```
Current state (RoadTwin, Part 35)
+ historical trajectory (Urban Memory, Part 36 – past defects, repairs, recurrence)
+ traffic exposure (Part 17)
  ↓
Predicted deterioration risk
```

Candidate inputs: historical defects, repair history, traffic volume, heavy-vehicle frequency, waterlogging history, weather (where available), repeated observations, road age (where available), defect recurrence, and temporal degradation rate.

Status — FUTURE / Research. This module is explicitly not part of the MVP or the Full-PS baseline (Parts 72–73); it requires enough historical Urban Memory data to be meaningful, which the project will not have until after a period of real or simulated multi-pass operation. It is documented here so the data model (Part 50) retains the fields it would need, but no prediction is claimed until it is trained and validated.

44. Adaptive Edge Intelligence

Cross-references the fast-path/slow-path model introduced in Part 12. Repeated here as its own numbered section per the required structure, since it spans both the edge architecture and the research track (Part 69).

FAST PATH → cheap detection/scene analysis, every scheduled frame
↓ uncertainty / event trigger (Part 28)
SLOW PATH → high-resolution model, tracking, segmentation, OCR, additional evidence

Driver of slow-path activation	Source
Scene complexity	Fast-path output entropy/variance
Uncertainty	Part 28 uncertainty classification
Road-segment importance	RoadTwin metadata — e.g., school zone, arterial route (Part 35)
Previous observations	Urban Memory — is this segment already CONFIRMED, or still ambiguous? (Part 36)
Safety risk	VRU/incident risk signals (Parts 21–22)
Incident probability	Recent trajectory-anomaly signals (Part 23)
Camera visibility	Observation-quality inputs (Part 27)

This reduces compute, energy, latency and bandwidth versus running every heavy model on every frame, while preserving detection quality on frames that matter most — the exact routing function is staged (Part 45) rather than assumed to be solved from day one.

45. Active Sensing

Formalizes Innovations E and P — which camera, and which bus, should receive computational/reporting priority right now.

Front camera: high road-defect probability
Rear camera: possible hit-and-run event
Side camera: pedestrian near moving vehicle
Cabin: passenger safety event

A prioritization function combines risk, uncertainty, information value, and computational cost:

$\text{priority}(\text{camera}, \text{task}) = f(\text{risk}, \text{uncertainty}, \text{information value}, -\text{compute cost})$

Tier	Formulation	Status
MVP heuristic	A simple rule table: incident-type triggers always take top priority; otherwise round-robin/weighted scheduling across cameras by recent detection rate.	PROPOSED for MVP
Advanced mathematical formulation	Explicit weighted scoring function per camera/task combining the four factors above, tuned via offline evaluation.	PROPOSED for Full PS
Research version	Mutual-information-based active sensing — trigger high-fidelity processing on an approaching bus specifically to maximize expected information gain about an uncertain road-state hypothesis $I(s;z) - \lambda \cdot \text{cost}$, see Part 69).	Research track

This does not require a mathematically sophisticated solution for the first implementation — the MVP heuristic is sufficient to demonstrate bandwidth/compute savings; the research version is reserved for the isolated paper (Parts 65–71).

46. GIS

Covers PS requirements 24, 25, 26 (Part 4.4). Twenty layers, expanded from v1.0's seven:

#	Layer	Content
1	Road health	Current defect/condition state per segment (Part 35)
2	Potholes	Confirmed/candidate potholes (Part 14.1)
3	Road damage	Broader surface-damage regions (Part 14.2)
4	Waterlogging	Waterlogging events, time-correlated (Part 14.3)
5	Infrastructure deficiencies	Missing/damaged signs, dividers, crossings (Part 15)
6	Traffic density	Per-segment density/flow (Part 17)
7	Congestion	Congestion heatmap (Part 17)
8	Bottlenecks	Flagged bottleneck segments (Part 18)
9	Route delays	Route/segment travel-time delay (Part 19)
10	Incidents	Rash-driving/hit-and-run markers (Part 22)
11	ANPR-restricted incident evidence	Access-controlled overlay, authorized roles only (Part 49)
12	Pedestrian-risk zones	VRU risk scoring output (Part 21)
13	School zones	Geofenced high-sensitivity zones (Part 21)
14	Maintenance	Queued/in-progress maintenance tasks (Part 38)
15	Repair verification	Verification-pending / verified-repaired markers (Part 39)
16	Reappeared defects	Recurrence markers (Part 40)
17	Fleet positions	Live/simulated bus positions and routes
18	Sensing coverage	Coverage Intelligence layer (Part 42)
19	Sensor health	Per-device/camera health status (Part 41)
20	Historical RoadTwin states	Time-scrubbing view of past states, repairs, recurrences (Part 36)

Integration: MapLibre or Leaflet on the frontend, rendering vector/GeoJSON layers served from PostGIS queries via the FastAPI backend. Exact mapping library choice remains OPEN DECISION (Part 86).

47. Authority Workflows

The dashboard is not merely a visualization — each authority type has a defined action path.

```
Traffic authority
Bottleneck (Part 18)
↓
Investigate
↓
Traffic trend (Part 17)
↓
Route delay (Part 19)
↓
Action
Roads department
Defect (Part 14/15)
↓
Priority (Part 38)
↓
Maintenance
↓
Repair
↓
Verification (Part 39)
Police / incident authority
Incident (Part 22)
↓
Vehicle track (Part 25)
↓
Plate evidence (Part 26)
↓
Location
↓
Secure investigation record
Transport authority
Route delay (Part 19)
↓
Fleet analysis
↓
Congestion (Part 17)
↓
Route planning insight
```

Each workflow terminates in either a persisted authority action (repair report, investigation record) or an analytics artifact (route planning insight) — the dashboard's job is to get an authorized user from a state to a decision, not just to render a map.

48. Alerts

Alert class	Trigger	Priority	Recipient role
Safety alert (VRU/incident)	Unsafe-interaction detection (Part 21) or incident trigger (Part 22)	P0 — real-time	Police/incident authority, roads department (for hazard cases)
Confirmed defect	Road Lifecycle CANDIDATE → CONFIRMED (Part 37)	P1/P2	Roads department
Missing infrastructure	Confident-absence declared on an expected asset (Part 15)	P2	Roads department
Repair verified / reappeared	Lifecycle transition into VERIFIED_REPAIRED or REAPPEARED (Parts 39–40)	P2	Roads department
Bottleneck/route delay	Sustained bottleneck/delay signal (Parts 18–19)	P2/P3	Traffic authority

Sensor health degraded	Sensor Health thresholds breached (Part 41)	P1 (operational)	Fleet/systems operator
------------------------	---	------------------	------------------------

Only unsafe-interaction and incident-class events are P0; routine infrastructure/traffic observations are batched per the bandwidth strategy (Part 59) rather than alerted individually.

49. Security & Privacy

Area	Requirement
Device authentication	Edge devices authenticate to the ingestion endpoint with device credentials.
API authentication	All API access authenticated; TLS for all HTTP traffic.
RBAC	Role-based access — e.g., ANPR/incident evidence restricted to authorized roles, distinct from general dashboard viewers (Part 46 layer 11).
Encrypted transport	TLS for HTTPS; authenticated, encrypted (TLS) MQTT rather than an open broker.
Evidence integrity	Evidence records should be tamper-evident (e.g., hashed) given their potential use in incident review.
ANPR restrictions	Plate data restricted to incident-review workflows; never exposed as general analytics or a public search capability (Part 49.1).
Cabin privacy	CAM_CABIN data scoped strictly to passenger-safety use cases (Part 11); excluded from road/traffic/ANPR pipelines.
Data retention	Retention windows for raw evidence vs. derived state should be explicitly defined — OPEN DECISION pending policy input.
Audit logs	Access to ANPR/incident evidence and any manual state overrides should be logged.
Tool authorization	Each Copilot tool call is scoped to the calling user's RBAC role (Part 52).
Prompt injection protection	Copilot tool calls are authorized and schema-constrained; retrieved/user text is never allowed to expand the Copilot's tool permissions.
Secure evidence access	Evidence media (crops/clips) served via access-controlled, time-limited references, not public URLs.

49.1 ANPR must never become general-purpose surveillance

ANPR is triggered only by an incident event (Part 22/26) — there is no mode in which the system runs plate recognition over ordinary passing traffic as a standing capability. This is a deliberate scope boundary, not an oversight, and is called out explicitly in Part 85.

50. Database Architecture

Conceptual entities, expanded to cover the full system (exact column-level schema remains an implementation task):

Entity	Purpose
BUS	A fleet vehicle (simulated identity in the prototype).
EDGE_DEVICE	The compute unit onboard a bus.
CAMERA	A physical/logical camera on a bus, with a role (Part 11).
ROUTE	A bus route, composed of an ordered set of road segments.
ROAD	A named road, composed of segments.
ROAD_SEGMENT	The atomic spatial unit RoadTwin state attaches to.
OBSERVATION	A single raw AI output before fusion.
EVENT	The transported record of an observation (Part 30).
VEHICLE	A detected vehicle instance with class (Part 16).
VEHICLE_TRACK	A tracked vehicle trajectory within/across frames (Part 25).
PLATE	An OCR'd license-plate result linked to a vehicle track (Part 26).

INCIDENT	A confirmed incident (rash driving, hit-and-run, etc.) with its evidence graph (Part 22.1).
ROAD_DEFECT	A defect record with lifecycle state (Part 37).
INFRASTRUCTURE_ASSET	A GIS-referenced expected asset (sign, divider, crossing) (Part 15.3).
EVIDENCE	Media/metadata backing an observation or event.
MAINTENANCE_TASK	A queued/assigned maintenance action (Part 38).
REPAIR	A reported repair action (Part 39).
VERIFICATION	The verification record/outcome for a repair (Part 39).
ALERT	A surfaced, authority-facing notification (Part 48).
ROADTWIN_STATE	The current persisted state snapshot per ROAD_SEGMENT (Part 35).
ROADTWIN_HISTORY	Time-series of past ROADTWIN_STATE snapshots — backs the historical GIS layer and freshness/decay logic (Parts 35.2, 46 layer 20).
MODEL	A registered AI model.
MODEL_VERSION	A specific trained/versioned instance of a MODEL — referenced by every EVENT for lineage.
USER	An authority/platform user account.
DEPARTMENT	An organizational unit a user belongs to.
SENSOR_HEALTH	Per-device/camera health records over time (Part 41).
COVERAGE	Aggregated observation-coverage records per segment (Part 42).
ROUTE_TRAVEL_TIME	Observed/expected travel-time records per route/segment (Parts 18–20).
TRAFFIC_OBSERVATION	Per-pass density/flow/speed readings feeding Traffic Intelligence (Part 17).

Key relationships: BUS 1—N EDGE_DEVICE—CAMERA; ROUTE N—N ROAD_SEGMENT; ROAD_SEGMENT 1—N OBSERVATION/EVENT/ROAD_DEFECT/INFRASTRUCTURE_ASSET/TRAFFIC_OBSERVATION/COVERAGE; ROAD_DEFECT 1—N EVIDENCE and 1—1 (current) ROADTWIN_STATE contribution, with ROADTWIN_STATE 1—N ROADTWIN_HISTORY; ROAD_DEFECT 1—1 MAINTENANCE_TASK 1—1 REPAIR 1—1 VERIFICATION; INCIDENT 1—N VEHICLE_TRACK and 0/1 PLATE; EVENT N—1 MODEL_VERSION.

51. API Architecture

Method	Endpoint	Purpose	Auth
POST	/api/v1/events	Ingest a canonical event from an edge device (Part 30)	Device credential
POST	/api/v1/evidence	Upload/register an evidence media reference	Device credential
POST	/api/v1/device/heartbeat	Edge device liveness signal	Device credential
GET	/api/v1/device/config	Fetch current runtime config/thresholds for a device	Device credential
GET	/api/v1/sensor-health	Query device/camera health history (Part 41)	User (RBAC)
GET	/api/v1/coverage	Query sensing-coverage map (Part 42)	User (RBAC)
GET	/api/v1/roadtwin	Query current RoadTwin state (filterable by area/segment)	User (RBAC)
GET	/api/v1/roadtwin/history/{segment_id}	Query historical RoadTwin states for a segment (Part 46 layer 20)	User (RBAC)
GET	/api/v1/road-segments/{id}	Fetch a specific segment's state and history	User (RBAC)
GET	/api/v1/road-defects	List/query road defects by status, area, severity	User (RBAC)
GET	/api/v1/infrastructure	Infrastructure-asset state query (Part 15)	User (RBAC)

GET	/api/v1/vehicles	Vehicle detection/classification/count query (Part 16)	User (RBAC)
GET	/api/v1/traffic	Traffic/density/congestion analytics query (Part 17)	User (RBAC)
GET	/api/v1/bottlenecks	Bottleneck query (Part 18)	User (RBAC)
GET	/api/v1/route-delay	Route-delay analytics (Part 19)	User (RBAC)
GET	/api/v1/od-analytics	Origin–destination approximation (Part 20 — FUTURE, honesty-flagged in response metadata)	User (RBAC)
GET	/api/v1/vru-risk	Pedestrian/VRU risk query (Part 21)	User (RBAC)
GET	/api/v1/incidents	List/query incidents	User (RBAC), restricted fields for ANPR
GET	/api/v1/incidents/{id}/evidence	Full evidence graph for one incident (Part 22.1)	User (RBAC), restricted
GET	/api/v1/anpr/{plate_id}	Restricted plate-evidence lookup (Part 26)	User (RBAC), restricted role only
GET	/api/v1/maintenance	List maintenance tasks/priority queue (Part 38)	User (RBAC)
POST	/api/v1/maintenance/{id}/repair	Authority reports a repair action taken	User (RBAC, authority role)
GET	/api/v1/verification	Query verification status/history for repaired defects (Part 39)	User (RBAC)
GET	/api/v1/alerts	Authority-facing alert feed (Part 48)	User (RBAC)
GET	/api/v1/fleet	Fleet/route/bus status	User (RBAC)
GET	/api/v1/analytics	General aggregate analytics (dashboards)	User (RBAC)
POST	/api/v1/copilot/query	Natural-language query to the GenAI Copilot (Part 52)	User (RBAC), tool-scoped

Error handling: all endpoints return a consistent error envelope (status code + machine-readable error type + human-readable message); RBAC failures return 403 rather than a silent empty result, so the frontend/Copilot can distinguish 'no data' from 'not authorized' (relevant for ANPR/incident-restricted endpoints).

52. GenAI Copilot

```

Natural language
↓
Intent parsing
↓
Authorized tool (RBAC-scoped, Part 49)
↓
Deterministic backend query (Part 51 API)
↓
Structured result
↓
LLM explanation

```

LLM CAN

- Interpret natural-language questions from authority users.
- Call authorized backend tools (RoadTwin query, analytics query, report generation).
- Explain structured results in plain language.
- Summarize evidence for a given segment/defect/incident, including its evidence lineage.

LLM CANNOT

- Invent road conditions not present in RoadTwin/backend data.
- Directly modify SQL or write to the database.
- Directly access unrestricted ANPR data (must go through access-controlled tools).
- Replace computer vision or PostGIS analytics.
- Decide maintenance priority without the deterministic backend logic (Part 38).

Example queries the Copilot must answer via tool calls, never invention

- "Which road segments have recurring potholes near schools?" — queries ROAD_DEFECT + INFRASTRUCTURE_ASSET (school zone) + recurrence history.
- "Show high-priority defects requiring attention." — queries the Part 38 maintenance-priority ranking.
- "Why was this road marked as high priority?" — retrieves the priority factor breakdown (Part 38), not a generated justification.
- "Which routes experienced abnormal delay today?" — queries Part 19 route-delay analytics.
- "Show incidents requiring investigation." — queries Part 22 with RBAC-appropriate evidence detail.

53. RAG

RAG (retrieval-augmented generation) is scoped to: maintenance SOPs, policies, official procedures, and safety guidelines — reference material that is genuinely document-shaped.

RAG is NOT the source for — traffic counts, road-state queries, fleet statistics, or any other numerical analytics. Those must always come from direct, deterministic queries against PostgreSQL/PostGIS via backend tools (Part 51), never from retrieved text.

54. Agentic Architecture

Option	Description	When it's the right call
1. Single LLM + tools	One model with a fixed toolset (RoadTwin query, analytics query, report generator).	Default recommendation for the Copilot — sufficient for authority Q&A and report generation.
2. Deterministic workflow + LLM	A fixed pipeline where the LLM only fills specific steps (e.g., summarization) inside an otherwise deterministic flow.	Good fit for structured tasks like automated maintenance-report generation.
3. Multi-agent system	Multiple cooperating/negotiating LLM agents.	Not recommended for this system — no requirement here needs agent-to-agent negotiation, and it adds failure surface without a clear benefit.

Recommendation: Option 1, with Option 2's pattern used specifically for report generation. Multi-agent architecture is not adopted merely because it is currently fashionable (Part 7.5's innovation admission rule) — it would only be reconsidered if a genuine coordination problem emerged that a single tool-using agent could not handle, and none has.

55. Dataset Strategy

Capability	Dataset	Source	Purpose	Data type	Limitations
Indian vehicles	IDD (India Driving Dataset)	Public	Realistic Indian vehicle appearance/behavior for detection	Public data	May not cover the exact routes/lighting of the demo city
Indian road scenes	IDD	Public	Realistic unstructured Indian road backgrounds	Public data	Geographic coverage may not include Hyderabad specifically
Vehicle detection	IDD + BDD100K	Public	Broaden vehicle-detection generalization beyond one dataset's bias	Public data	Domain gap between BDD100K (largely non-Indian) and Indian traffic
Vehicle tracking	BDD100K MOT	Public	Multi-object tracking training/validation	Public data	Camera motion patterns differ from a bus platform
Pedestrians	IDD + BDD100K	Public	Pedestrian detection across varied traffic density	Public data	Indian dense/unstructured pedestrian patterns underrepresented in BDD100K
Road segmentation	IDD	Public	Road-surface segmentation for defect localization	Public data	Not necessarily annotated for defect-specific classes
Potholes / road damage	Road Damage Dataset + custom data	Public + custom	Defect detection training	Public + custom-collected	Public sets vary by country's road material/appearance
Traffic signs	Public traffic-sign dataset + custom India data	Public + custom	Sign detection/condition classification	Public + custom-collected	Indian sign standards differ from many public sign datasets
Waterlogging	Custom dataset + suitable public images	Custom + public	Waterlogging detection	Custom-collected + public	No large standardized public waterlogging benchmark exists
ANPR	Public plate data + custom footage	Public + custom	Plate detection/OCR training	Public + custom-collected	Indian plate formats/fonts vary; needs targeted custom data
Incident tracking	Custom controlled footage	Custom	Rash-driving/hit-and-run scenario data	Custom-collected	Real incidents are rare and hard to source publicly — must be simulated/staged
GPS	Own recorded GPS traces	Custom	Location association, map matching	Custom-collected (simulated fleet)	Not live production telemetry
Bus fleet simulation	Own video + GPS + route metadata	Custom / synthetic	End-to-end demo simulation	Simulated	Explicitly a simulation, not a live deployment
GIS / infrastructure inventory	Generated from own GPS/events + manual annotation	Derived + custom	Reference layer for infrastructure/road segments (Part 15.3)	Derived/synthetic	Coverage limited to demo route unless expanded

IDD's multimodal release is particularly useful where synchronized video/GPS-style data is needed for experimentation.

56. Indian Domain Adaptation

- Auto-rickshaws and other vehicle classes rarely present in Western datasets.
- Motorcycles/two-wheelers at far higher density and with different lane behavior.
- Dense, unstructured traffic without strict lane discipline.
- Road markings that are faded, inconsistent, or absent.
- Indian plate formats/fonts differing from datasets OCR models are typically tuned on.
- Indian-specific signage standards.
- Road-surface materials and degradation patterns differing from other countries' road-damage datasets.

Practical fine-tuning strategy

- Start from a strong pretrained detector/tracker (e.g., a public COCO/BDD-pretrained backbone) rather than training from scratch.
- Fine-tune on IDD + custom Hyderabad-area footage for vehicle/pedestrian classes specific to Indian traffic, including auto-rickshaw and motorcycle-density handling (Part 16).
- Fine-tune OCR specifically on Indian plate samples (font, layout, state codes).
- Use a held-out, geographically-separated validation split so fine-tuning gains are not just memorizing the same streets (Part 63).

57. ML Training Pipeline

Dataset → Cleaning → Annotation → Train/Validation/Test split
 → Transfer Learning → Fine-tuning → Evaluation → Error Analysis
 → Model Registry → ONNX export → Edge Runtime

- Experiment tracking — every training run's config/metrics logged (tool choice OPEN DECISION).
- Model versioning — each deployed model tied to a MODEL_VERSION record (Part 50), enabling the lineage requirement (Part 33.1).
- Reproducibility — training configs and random seeds recorded alongside results.
- Data versioning — dataset splits/versions tracked so a reported metric is always traceable to the exact data it was measured on.
- Validation — geographically/temporally separated validation to avoid optimistic leakage (Part 63).

58. Edge Optimization

Technique	Tier
Frame sampling (not every frame needs full inference)	MVP
Dynamic frame scheduling across cameras (Part 11)	MVP
Fast-path/slow-path routing (Part 12/44)	MVP heuristic → Advanced formulation (Part 45)
Selective OCR (only on incident trigger, Part 26)	MVP
Selective tracking (only when relevant classes present)	MVP
Model quantization (INT8/FP16)	Advanced
ONNX export + ONNX Runtime execution providers	MVP (this is the HAL foundation, Part 10)
TensorRT optimization on Jetson-class hardware	Advanced / future deployment
Information-gain-triggered processing (Part 45 research tier)	Research
CPU vs. GPU execution selection by device profile	MVP (via execution-provider config)

59. Bandwidth Strategy

	Continuous raw video	Event-driven evidence transmission (chosen approach)
Data volume	Very high — full-resolution streams per camera, continuously	Low — small JSON events + compressed crops, only when significant

Cellular cost	High and roughly constant regardless of road condition	Scales with actual events, not with time driven
Central storage	Requires storing/streaming raw video at scale	Stores only evidence crops/clips tied to confirmed events
Real-time viability	Poor — congests uplink, especially P0 safety cases	Good — small P0 payloads prioritized over MQTT/HTTPS

- Event metadata — the canonical JSON event (Part 30), always small.
- Evidence frame — a single best-quality crop supporting the event.
- Cropped evidence — spatially cropped rather than full-frame, where possible.
- Video clip — reserved for incident evidence packages (Part 22), not routine defect events.
- Compression — standard image/video compression on all evidence payloads before transmission.
- Queue — local edge queue for events pending transmission.
- Retry — automatic retry with backoff on transmission failure.
- Offline mode — edge agent continues detecting/queuing when connectivity is lost, syncing once restored.

Dual-priority routing (PROPOSED production refinement, Part 31): real-time P0/P1 safety events go out immediately over cellular MQTT/HTTPS as small payloads; bulkier P2/P3 routine infrastructure logs can be batched and synced opportunistically (e.g., over depot Wi-Fi) to avoid cellular cost. No specific bandwidth or cost figures are quoted here; those must be measured, not assumed.

60. Failure Modes

Merges the v1.0 failure-mode table with the upgrade brief's explicit failure-aware design requirement: the system must always distinguish 'no defect detected' from 'no valid observation available' (Part 34's negative-evidence gate is the mechanism).

Failure	Cause	Impact	Detection	Mitigation
Poor lighting	Night/low-light conditions	Reduced detection accuracy	Illumination component of quality score (Part 27) drops	Weight low-quality observations down in fusion; avoid confirming on low-quality evidence alone
Rain	Weather-induced visual degradation	False negatives/positives for defects and waterlogging	Quality score + weather correlation flag	Require higher corroboration during flagged weather
Motion blur	Bus speed/vibration	Missed or malformed detections	Blur component of quality score	Discount confidence contribution; prefer sharper frames from the multi-frame buffer
Occlusion	Other vehicles/pedestrians blocking view	False negative for defect/infrastructure presence	Occlusion estimate feeding the negative-evidence gate (Part 34)	Do not count occluded passes as confident absence
Camera failure	Hardware/stream fault	Missing observations misread as 'clean road'	Sensor Health (Part 41)	Flag device unhealthy; exclude its passes from fusion until restored
GPS drift	Consumer-grade or simulated GNSS inaccuracy	Misattributed road segment	Accuracy field from LocationProvider (Part 10)	Map-matching tolerance + multi-pass consistency check
Model failure	Inference crash/exception on edge	Missed frame(s)	Exception logging + Sensor Health	Fail-safe skip-and-continue; do not crash the whole agent
Duplicate events	Same physical event reported multiple times (retry, multi-camera overlap)	Inflated corroboration count	Deduplication by segment+time+type+device (Part 32)	Event-level deduplication before fusion input

Delayed events	Network/queue backlog	Stale-looking data at ingestion time	Event timestamp vs. ingestion timestamp comparison	Process on event timestamp, not arrival order
Conflicting buses	Two buses report contradictory observations for the same segment/time	Ambiguous fusion result	Fusion engine detects disagreement magnitude (Part 33.1)	Weight by source reliability/quality; flag for lower-confidence state rather than forcing a decision
Incorrect map matching	GPS/segment-boundary ambiguity	Observation attributed to wrong segment	Map-matching confidence score	Require minimum match confidence before attaching to a segment
Stale observations	No recent re-check of a segment	State trusted longer than warranted	Freshness/decay tracking (Part 35.2)	Decay confidence; mark STALE distinctly
False absence	Confident-absence declared incorrectly	Premature VERIFIED_REPAIRED	Post-hoc reappearance detection (Part 40)	Conservative absence-confidence threshold (Part 37)
False confirmation	CONFIRMED declared on insufficient evidence	Wasted maintenance dispatch	Audit of confirmation evidence trail (Part 42)	Require quality-gated corroboration, not confidence alone (Part 27)
Network loss	Cellular/backhaul unavailability	Delayed event delivery	Missed heartbeat	Offline mode + queue + retry (Part 59)
Backend failure	Service outage	Events not processed	Health checks (Part 61)	Durable ingestion queue; retry from edge

61. Observability

- Logs — structured logs at both edge and backend.
- Metrics — per-module performance metrics (latency, throughput, error rate).
- Health checks — backend service health endpoints.
- Heartbeats — periodic edge-device liveness signal (Part 10 `EventTransport.publish_heartbeat()`).
- Event tracing — an event should be traceable end-to-end from edge capture to RoadTwin update (Part 42).
- Model metrics — inference latency/confidence distribution monitored over time.
- Latency monitoring — end-to-end and per-stage.
- Queue monitoring — edge/backend queue depth and retry rates.
- Device health — per-bus/device status visible centrally (Part 41).

62. Testing

Test type	Scope
Unit tests	Individual functions/classes per module (detectors, fusion rules, state-machine transitions).
Integration tests	Module boundaries — e.g., edge event schema correctly consumed by ingestion.
Model tests	Detector/tracker/OCR accuracy on held-out validation data.
API tests	Every endpoint in Part 51 against expected request/response contracts.
Database tests	Schema constraints, relationship integrity (Part 50).
GIS tests	Correct rendering of each of the 20 layers (Part 46) against known test data.
End-to-end tests	Full pipeline: recorded video → edge AI → event → fusion → RoadTwin → dashboard.
Failure tests	Deliberate injection of the failure modes in Part 60.

Load tests	Multiple simulated buses/events to check ingestion and fusion scale reasonably.
Security tests	AuthN/AuthZ enforcement, ANPR access restriction, MQTT/API encryption (Part 49).
System acceptance tests	The 15 full-scenario tests in Part 64.
Demo tests	Dry-run of the exact Part 82 demo script before presenting to judges.

63. Evaluation

Category	Metrics
Computer Vision	Precision, Recall, F1, mAP
Tracking	ID consistency (ID-switch rate), tracking accuracy
Traffic	MAE, MAPE (density/flow/speed estimates), delay-detection accuracy
Road State	Confirmation precision, false-confirmation rate, missed-reappearance rate, state-transition accuracy
Uncertainty	Calibration, coverage (if conformal methods used), abstention quality
Coverage	Fraction of road network with strong/weak/no recent observation (Part 42)
Edge	FPS, latency, CPU/GPU utilization, memory, energy where measurable
Bandwidth	kbps/km, evidence payload size, reduction versus raw streaming

No specific target numbers are asserted here — all figures must come from actual benchmarking on the team's chosen hardware and datasets before being quoted to judges or reviewers (Part 85).

64. Full PS Acceptance Tests

Fifteen system-level scenarios. Each must define input, processing, expected event, expected RoadTwin state, expected dashboard result, expected alert, and an acceptance criterion before it counts as passing.

Scenario 1 — Pothole lifecycle

Aspect	Detail
Input	Recorded footage with a real pothole, played for two simulated buses (A then B) on the same segment
Processing	BUS A detects → event → CANDIDATE. BUS B corroborates on a later pass → CONFIRMED (Part 33.1). Authority reports repair → REPAIR_REPORTED → VERIFICATION_PENDING. A later pass shows confident absence → VERIFIED_REPAIRED. A further pass shows the defect again → REAPPEARED.
Expected event	pothole event_type, two independent instances, then absence-consistent passes, then a new positive event
Expected RoadTwin state	Full lifecycle traversal: OBSERVED→CANDIDATE→CONFIRMED→MAINTENANCE_PENDING→REPAIR_REPORTED→VERIFICATION_PENDING→VERIFIED_REPAIRED→REAPPEARED
Expected dashboard result	Road Health layer shows the correct status at each stage
Expected alert	Confirmed-defect alert on CONFIRMED; repair-verified alert on VERIFIED_REPAIRED; recurrence alert on REAPPEARED
Acceptance criterion	All seven lifecycle states are reached in the correct order with the correct triggering evidence

Scenario 2 — Waterlogging

Aspect	Detail
Input	Footage showing standing water persisting across a short time window, plus a control clip of a transient puddle
Processing	Persistence filter (Part 14.3) accepts the standing case and rejects the transient one
Expected event	waterlogging event only for the persistent case
Expected RoadTwin state	CANDIDATE→CONFIRMED for persistent case; no CANDIDATE created for the transient case
Expected dashboard result	Waterlogging layer shows only the persistent instance
Expected alert	P2 alert to roads department
Acceptance criterion	Transient puddle never reaches CANDIDATE; persistent waterlogging reaches CONFIRMED

Scenario 3 — Missing sign

Aspect	Detail
Input	GIS reference states a sign should exist at segment X; footage shows the location clearly visible with no sign present, across 3 independent passes
Processing	Expected-vs-observed comparison (Part 15) gated by the negative-evidence checks (Part 34)
Expected event	missing_sign event with comparison confidence
Expected RoadTwin state	Infrastructure state = ABSENT after 3rd independent high-quality pass
Expected dashboard result	Infrastructure Deficiencies layer flags the segment
Expected alert	P2 alert to roads department
Acceptance criterion	A single occluded pass does not trigger ABSENT; 3 independent clear passes do

Scenario 4 — Traffic bottleneck

Aspect	Detail
Input	Simulated multi-bus GPS/speed data showing sustained low speed at one segment across several passes, vs. a control segment with a one-off slow vehicle
Processing	Bottleneck logic (Part 18) requires persistence across passes/buses
Expected event	traffic_observation events aggregated centrally
Expected RoadTwin state	Bottleneck flag set on the persistent segment only
Expected dashboard result	Congestion/Bottleneck layer highlights the correct segment

Expected alert	P2/P3 alert to traffic authority
Acceptance criterion	The one-off slowdown does not trigger a bottleneck flag; the sustained case does

Scenario 5 — School pedestrian risk

Aspect	Detail
Input	Footage of a child near the roadway inside a geofenced school zone with a vehicle closing distance, vs. a control clip of a pedestrian far from the road
Processing	VRU trajectory + risk scoring (Part 21) inside the school-zone geofence
Expected event	pedestrian_risk event only for the closing-distance case
Expected RoadTwin state	Incident state reflects a risk event, not a plain pedestrian detection
Expected dashboard result	Pedestrian-Risk / School-Zone layers show the flagged case
Expected alert	P0 safety alert for the closing-distance case only
Acceptance criterion	Plain pedestrian presence never generates a P0 alert by itself; genuine unsafe interaction does

Scenario 6 — Rash driving

Aspect	Detail
Input	Footage/simulated trajectory with a sudden erratic lane change and abnormal closing speed
Processing	Trajectory/kinematic anomaly detection (Part 23) triggers the rolling evidence buffer (Part 22)
Expected event	incident event (rash_driving) with pre/post buffer
Expected RoadTwin state	INCIDENT record created with evidence graph (Part 22.1)
Expected dashboard result	Incident layer shows the event
Expected alert	P0 alert to police/incident authority
Acceptance criterion	Pre-event buffer frames are present in the evidence package, not just post-event frames

Scenario 7 — Hit-and-run + ANPR

Aspect	Detail
Input	Footage of a simulated collision/near-collision followed by the vehicle leaving frame, with a legible plate at some point
Processing	Incident trigger → vehicle tracking (Part 25) → ANPR (Part 26) with all three confidence values recorded
Expected event	incident event + anpr event linked via the evidence graph
Expected RoadTwin state	INCIDENT + PLATE records with vehicle-detection, plate-detection and OCR confidences all present and distinct
Expected dashboard result	Incident layer (restricted ANPR overlay for authorized roles)
Expected alert	P0 alert to police/incident authority
Acceptance criterion	All three confidence values are present and different from each other (not collapsed into one number)

Scenario 8 — Network outage

Aspect	Detail
Input	Edge agent loses connectivity for a defined window while continuing to detect events
Processing	Offline queueing (Part 59) buffers events locally; retries once connectivity returns
Expected event	All events eventually arrive, timestamped at capture time not arrival time
Expected RoadTwin state	RoadTwin updates reflect capture-time ordering once events arrive
Expected dashboard result	No dashboard gap beyond the actual outage window, once synced
Expected alert	Delayed-delivery flag on affected events, no false 'no activity' alert
Acceptance criterion	Zero events are lost; all are processed in correct temporal order after reconnection

Scenario 9 — GPS drift

Aspect	Detail
Input	Simulated GPS trace with an injected positional error near a segment boundary
Processing	Map-matching confidence check (Part 60) flags low-confidence matches
Expected event	Event map_match_confidence below threshold

Expected RoadTwin state	Observation held pending re-matching rather than silently assigned to the wrong segment
Expected dashboard result	No incorrect segment update visible on GIS
Expected alert	No alert (this is a data-quality case, not a road condition)
Acceptance criterion	The observation is never attached to the wrong ROAD_SEGMENT

Scenario 10 — Camera failure

Aspect	Detail
Input	One camera stream deliberately stopped mid-run
Processing	Sensor Health (Part 41) detects the stream stopping and flags the device unhealthy
Expected event	sensor_health event (camera_unavailable)
Expected RoadTwin state	Segment observations from that camera during the outage window are excluded from negative-evidence calculations (Part 34)
Expected dashboard result	Sensor Health layer shows the camera as unhealthy for the affected window
Expected alert	P1 operational alert to fleet/systems operator
Acceptance criterion	No false 'confident absence' is produced from frames the camera never actually captured

Scenario 11 — Conflicting bus observations

Aspect	Detail
Input	BUS X reports a defect at a segment; BUS Y, passing minutes later under good visibility, reports no defect
Processing	Fusion engine (Part 33.1) weighs disagreement by quality/source reliability rather than forcing an immediate decision
Expected event	Both observations retained, linked to the same candidate
Expected RoadTwin state	State remains CANDIDATE (not CONFIRMED, not dismissed) pending further evidence
Expected dashboard result	Dashboard shows the segment as still under review, not falsely confirmed or falsely cleared
Expected alert	No premature alert
Acceptance criterion	The system does not silently pick one bus's report as 'the truth' without further corroboration

Scenario 12 — Duplicate event

Aspect	Detail
Input	Two cameras on the same bus, or a retried transmission, report the same physical pothole in the same pass
Processing	Deduplication (Part 32) collapses these into a single observation before fusion
Expected event	One deduplicated event stored, not two
Expected RoadTwin state	Corroboration count is not inflated by the duplicate
Expected dashboard result	Road Health layer shows a single defect instance
Expected alert	No duplicate alert
Acceptance criterion	Corroboration count used for CONFIRMED reflects genuinely independent passes only (Part 33.1)

Scenario 13 — False absence

Aspect	Detail
Input	A defect is prematurely marked VERIFIED_REPAIRED from a single ambiguous pass (a deliberately injected test case), then a later high-quality pass detects it again
Processing	Reappearance detection (Part 40) catches the corrected case
Expected event	New positive event at a VERIFIED_REPAIRED segment
Expected RoadTwin state	State transitions VERIFIED_REPAIRED → REAPPEARED
Expected dashboard result	Dashboard reflects the correction, not a silent contradiction
Expected alert	Recurrence alert to roads department
Acceptance criterion	The system corrects itself via new evidence rather than requiring manual database intervention

Scenario 14 — Route delay

Aspect	Detail
--------	--------

Input	Simulated bus trajectory on a known route showing travel time substantially above the route's baseline
Processing	Route-delay analytics (Part 19) compares observed vs. baseline travel time
Expected event	route_travel_time observation aggregated centrally
Expected RoadTwin state	ROUTE_TRAVEL_TIME record flags the delay
Expected dashboard result	Route Delay layer highlights the affected route/segment
Expected alert	P2/P3 alert to transport authority
Acceptance criterion	The delay is only flagged when it exceeds the baseline by a defined margin, not on every minor variation

Scenario 15 — Authority Copilot query

Aspect	Detail
Input	An authority user asks: "Which road segments have recurring potholes near schools?"
Processing	Copilot parses intent → calls the authorized RoadTwin/defect-recurrence/school-zone tool (Part 52) → backend returns structured results → Copilot explains them
Expected event	No new event — this is a read-only query
Expected RoadTwin state	No RoadTwin state change
Expected dashboard result	Copilot response matches what a direct API query (Part 51) would return
Expected alert	No alert
Acceptance criterion	Every fact in the Copilot's answer is traceable to an actual backend query result, not generated text

65. Research Question

"Can lifecycle-aware probabilistic evidence fusion using asynchronous fleet observations improve road-state estimation and repair verification compared with independent frame-level detections?"

This isolates the fusion/lifecycle/negative-evidence model (Parts 33–34, 37–40) from the rest of the engineering system (Part 35, Research/Engineering Separation Table) as the paper's falsifiable claim.

66. Research Hypotheses

- H1 — A lifecycle-aware fusion model that incorporates negative evidence achieves higher repair-verification precision than independent frame-level detection or simple temporal smoothing.
- H2 — Explicitly modeling observation quality (Part 27) alongside model confidence reduces false-confirmation rate compared with confidence-only thresholding.
- H3 — Genuinely independent multi-bus corroboration (Part 33.1) reduces false-confirmation rate more than repeated single-bus/single-camera observations of the same count.
- H4 — Spatial entropy over a segment's true state decreases as the number of independent asynchronous bus traversals increases (the theoretical convergence claim underlying the POMDP framing, Part 69).

Each hypothesis must be tested against the baselines and ablations below before being reported as supported — none is asserted as already confirmed.

67. Research Baselines

Baseline	Description
1. Independent detector	Raw YOLO-class detections, no temporal or cross-bus fusion.
2. Temporal smoothing	Simple moving-average/majority-vote across a single bus's repeated passes.
3. Simple cross-bus corroboration	Count-based corroboration across buses without quality weighting or negative-evidence handling.
4. Classical truth discovery	Established truth-discovery algorithms (e.g., RATD, CRH) applied to the same observation stream, so reviewers do not read this as an incremental application paper.
5. Proposed — lifecycle-aware fusion	The full model in Parts 33–34 and 37, including quality weighting, independence checking, and negative evidence.

Additional academic reference points noted in the research-strategy discussion: temporal spatial-GNN baselines and Extended Kalman/particle filters, included where feasible given time constraints.

68. Ablation Studies

- No negative evidence — remove the Part 34 gate; treat every non-detection as absence.
- No observation quality — use raw model confidence only (Part 27 removed).
- No cross-bus evidence — restrict fusion to single-bus repeated passes only.
- No temporal persistence — require only a single observation for CANDIDATE→CONFIRMED.
- No uncertainty — force a binary confirm/reject decision, removing the abstain/candidate states (Part 28).
- No lifecycle model — collapse the full state machine (Part 37) into a single 'detected' flag.

Each ablation isolates exactly which component of the proposed model drives any observed improvement over the baselines.

69. Mathematical Formulation

Two theoretical anchors were proposed; the team selects one as primary (OPEN DECISION, Part 86).

- POMDP / Dynamic Factor Graph framing — model the road network as a continuous-space Partially Observable Markov Decision Process. Hidden state transitions follow a stochastic transition matrix $P(s[t+1] \mid s[t], \Delta t)$, capturing physical decay as a function of elapsed time and traffic volume. The observation likelihood $P(z[t] \mid s[t], c[t])$ is conditioned on a context vector $c[t] = [\text{velocity, illuminance, occlusion ratio, camera angle}]$. A target theoretical result would show spatial entropy $H(s[t])$ shrinking toward zero as the number of asynchronous bus traversals N grows (H4 above).
- Information-theoretic active sensing — frame opportunistic high-fidelity ('slow-path', Part 44) processing as maximizing mutual information $I(s[t]; z[t+1])$ about an uncertain road state against the compute/energy cost $C(a[t])$ of triggering it: choose $a[t] \in \{0,1\}$ to maximize $I(s[t]; z[t+1]) - \lambda \cdot C(a[t])$ (Part 45 research tier).

Uncertainty calibration for the detector itself (Evidential Deep Learning with Dirichlet output heads, and/or split conformal prediction for distribution-free coverage guarantees) is a further refinement layered on top of either anchor — Research tier (Part 28), not required for the SIH engineering demo.

70. Research Contribution

Category	What belongs here
Engineering contribution	The working end-to-end pipeline: edge HAL, event schema, ingestion, GIS dashboard (Parts 10–13, 32, 46).
System contribution	The overall architecture pattern connecting edge sensing to a persistent city-scale state layer (RoadTwin, Parts 35–37).
AI contribution	Applying detection/tracking/OCR to the Indian-traffic domain with appropriate fine-tuning (Part 56).
Potential research contribution	The lifecycle-aware, uncertainty-driven, quality-aware evidence-fusion model, isolated and evaluated against the baselines in Part 67.

71. Novelty Validation

71.1 Existing technologies ("no fake novelty")

YOLO, OCR, ANPR, tracking, GIS, MQTT, FastAPI, PostGIS, edge computing, RAG, and LLMs are established technologies; none is individually novel, and none of the 20 innovations in this document (Parts 33.1, 34, 41–45, and elsewhere) claims otherwise.

71.2 Potential UrbanSense contribution

To be investigated and experimentally validated: lifecycle-aware, uncertainty-aware, quality-aware evidence fusion over asynchronous mobile observations, including principled negative evidence for repair verification and recurrence detection. This is stated as a hypothesis (Part 66), not asserted as confirmed novelty until a literature review and the experiments in Parts 67–68 are complete.

71.3 Novelty Opportunity Matrix

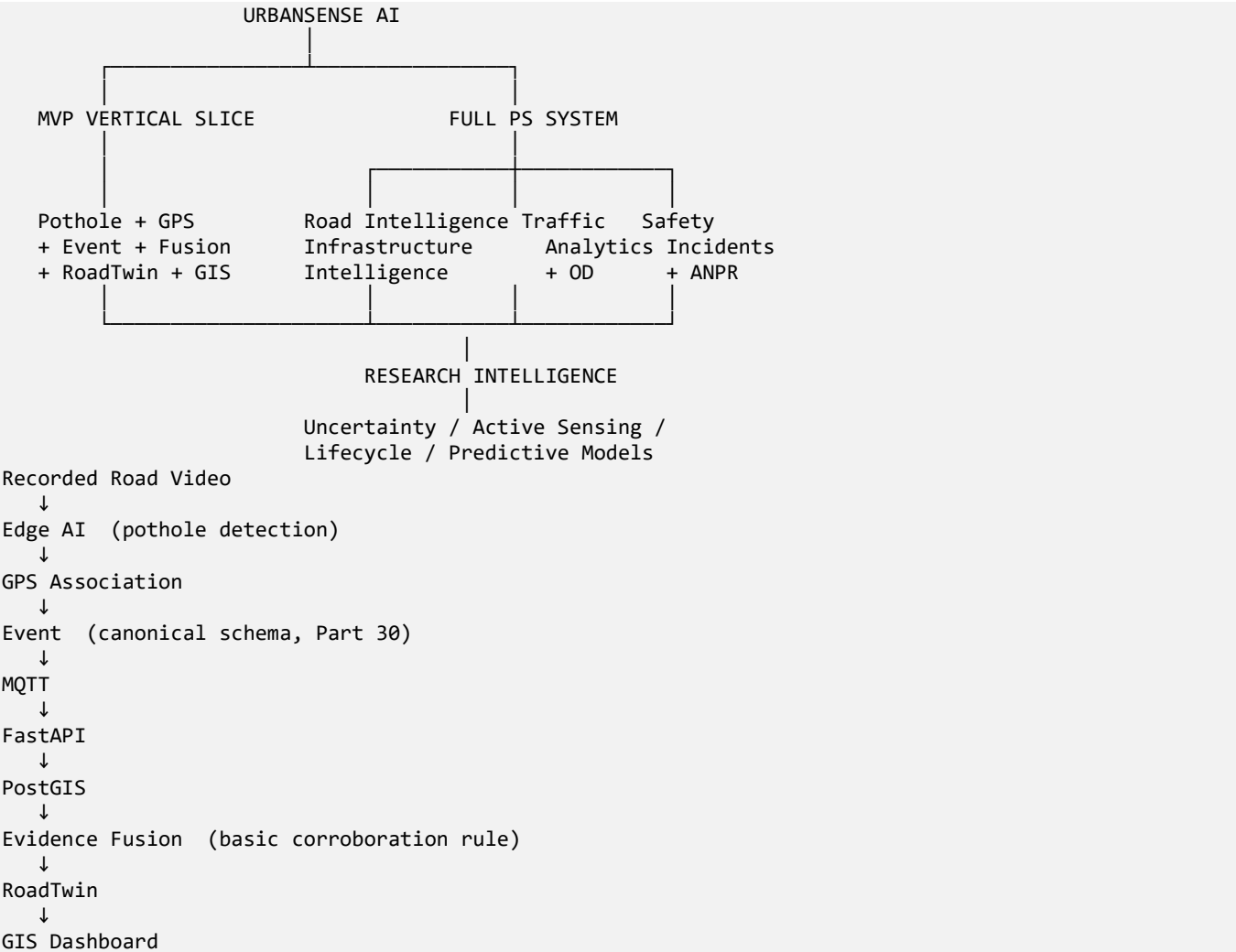
Innovation	Existing research?	UrbanSense adaptation	Difficulty	SIH value	Research value	Priority
Lifecycle-aware road state (37,39,40)	Partial — defect detection is common; lifecycle/repair-verification modeling is less so	Full OBSERVED→REAPPEARED state machine tied to a real maintenance workflow	Medium	High	High	1
Quality-aware negative evidence (34)	Related to sensor-fusion 'absence of evidence' literature; underused in road sensing	Explicit multi-gate visibility/occlusion/blur check before declaring absence	Medium	High	High	1

Cooperative mobile sensing, independence-aware (33.1)	Related to crowdsensing/truth-discovery literature	Explicit independent-vs-duplicate-vs-correlated classification for road evidence	Medium	Medium	Medium-High	2
Adaptive edge (fast/slow path) (12,44)	Well-studied in general CV/edge literature (cascade detectors)	Applied specifically to multi-camera bus-mounted road sensing	Low-Medium	High (feasibility story)	Low-Medium	2
Active/risk-aware sensor scheduling (45)	Related to active-learning/active-sensing literature	Mutual-information framing applied to opportunistic bus passes	High	Low (not needed for demo)	High	3 (research-only)
Coverage intelligence (42)	Related to sensing-coverage mapping in mobile-sensor-network literature	Applied to a bus-fleet infrastructure-inspection context	Low	Medium	Medium	2
Predictive deterioration (43)	Studied in pavement-engineering literature, typically with richer sensor/maintenance data	Adapted to fleet-only observation history	High (needs data we don't yet have)	Low (not implementable now)	Medium (future)	4 (future)
Cross-bus vehicle continuity (24.1)	Related to multi-camera re-identification literature	Applied probabilistically, not claimed as re-identification	High	Low	Medium	3 (research-only)

Priority guides where the team invests limited time: items marked '1' (lifecycle state + negative evidence) are both the strongest SIH differentiators and the strongest research candidates, and are treated as required for the Full PS build (Part 73), not optional research extras.

72. MVP

Redefinition — MVP means the first complete vertical slice that proves the architecture works end to end. It does NOT mean the remaining PS-26124 requirements are optional. Full PS coverage (Part 73) remains mandatory; only the order of implementation is staged.



Must be implemented for MVP: FileCameraProvider + SimulatedGNSSProvider; the pothole detector; the canonical event schema end-to-end; a minimal fusion rule (a simple corroboration-count threshold is acceptable at MVP, upgraded per Part 33.1 as the system matures); RoadTwin persistence for at least OBSERVED→CONFIRMED; a basic map view showing confirmed defects. The MVP proves the architecture — it is deliberately narrow in AI-capability breadth, not in the correctness of the pipeline.

73. Full PS Implementation

Every capability required by PS 26124 (Part 3), organized by the module that delivers it. This is the scope the team commits to building, staged across the roadmap (Part 75) but never permanently deferred to 'future work.'

Capability group	Modules delivering it	Required by
Road defects (potholes, damaged roads, waterlogging, other hazards)	Road Defect Intelligence (14)	PS 3.2
Missing/damaged infrastructure (dividers, zebra crossings, signs)	Infrastructure Intelligence (15)	PS 3.2, 3.3

Vehicle detection/classification/counting, traffic density	Vehicle Intelligence (16), Traffic Intelligence (17)	PS 3.2
Bottleneck detection	Bottleneck Detection (18)	PS 3.2
Vulnerable pedestrians / school-child risk	VRU Intelligence (21)	PS 3.2
Rash driving / hit-and-run + offending-vehicle tracking + ANPR + confidence/timestamp/GPS	Incident Intelligence (22), Rash Driving (23), Hit-and-Run (24), Vehicle Tracking (25), ANPR/OCR (26)	PS 3.2
Secure alerting	Secure Transport (31), Alert Engine (48)	PS 3.2
Centralized fleet aggregation	Central Platform (32), Evidence Fusion (33)	PS 3.3
GIS visualization, congestion heat maps	GIS (46), Traffic Intelligence (17)	PS 3.3
Infrastructure-deficiency analysis	Infrastructure Intelligence (15)	PS 3.3
Origin–destination analysis, route-delay estimation	OD Analytics (20), Route Delay (19)	PS 3.3
Actionable transport-authority insights	Maintenance Intelligence (38), Authority Workflows (47)	PS 3.3
Edge-AI processing, bandwidth minimization	Hardware Abstraction Layer (10), Adaptive Edge (44), Bandwidth Strategy (59)	PS 3.4

Cross-check against Part 4's Requirement Traceability Matrix confirms zero unexplained PS requirements.

74. Advanced Research Track

Capabilities that go beyond the minimum PS requirement, staged as Research or Future per Part 7.5's innovation admission rule — not required for SIH judging, but strengthening the platform's credibility and the paper's depth.

- Sensor Health (41) and Coverage Intelligence (42) — self-diagnosis and coverage mapping beyond what PS 26124 explicitly asks for, but directly supporting trustworthy negative evidence (Part 34).
- Predictive Road Intelligence (43) — explicitly FUTURE, pending sufficient Urban Memory history.
- Active Sensing (45) — MVP heuristic and advanced formulation are in-scope for the Full PS build; the mutual-information research formulation is Research-tier only.
- Cross-camera/cross-bus vehicle continuity (24.1) — Research-tier, framed as probabilistic continuity, not re-identification.
- Evidential Deep Learning / conformal prediction (28) — Research-tier calibration layer on top of the Full PS detector stack.
- The full mathematical formulation and baseline/ablation program (Parts 65–69) — reserved for the isolated academic paper, not the SIH build.

75. Implementation Roadmap

Phase	Focus
0	Repository, Docker, development environment
1	HAL + video/GPS ingestion (Part 10)
2	Core perception — detection models (Parts 13–14, 16)
3	Tracking (Part 25)
4	Road-defect intelligence (Part 14)
5	Infrastructure intelligence (Part 15)
6	Traffic intelligence, bottleneck, route-delay signals (Parts 17–19)
7	VRU/safety intelligence (Part 21)
8	Incident intelligence, rash driving, hit-and-run (Parts 22–24)
9	ANPR/OCR (Part 26)

10	Event transport, canonical schema, secure transport (Parts 30–31)
11	Backend/database (Parts 32, 50)
12	Evidence fusion, negative evidence (Parts 33–34)
13	RoadTwin / Urban Memory (Parts 35–36)
14	Maintenance lifecycle, repair verification, reappearance (Parts 37–40)
15	GIS — 20 layers (Part 46)
16	OD/route-delay analytics (Parts 19–20)
17	Alerts/security (Parts 48–49)
18	GenAI Copilot/RAG (Parts 52–53)
19	Advanced uncertainty/active sensing (Parts 28, 45)
20	Sensor health/coverage intelligence (Parts 41–42)
21	Full integration
22	Testing (Parts 62, 64)
23	Benchmarking (Part 63)
24	SIH demo (Part 82)
25	Research experiments (Parts 65–69)

Ordering may be optimized further once actual module dependencies are known; the sequence above respects the dependency that fusion/RoadTwin (Phases 12–13) need the event pipeline (Phase 10) and backend (Phase 11) first, and that the demo (Phase 24) needs integration and testing (Phases 21–22) complete.

76. Team Responsibilities

Member	Responsibility	Key technologies	Interfaces with
1 — Computer Vision	Road-defect, infrastructure, vehicle and pedestrian detection models; model training/fine-tuning (Parts 14–16, 21, 56–57)	YOLO-family detector, PyTorch, ONNX export	Sends observations to Member 3's event engine
2 — Tracking / Motion / Traffic	MOT, trajectories, traffic density, bottlenecks, route-delay signals, incident motion, rash-driving/hit-and-run kinematics (Parts 17–19, 23–25)	MOT frameworks	Consumes Member 1's detections; feeds Member 3's event/incident logic
3 — Edge / Systems	HAL, cameras, GPS/IMU, scheduling (fast/slow-path), ONNX, Docker, queues, bandwidth, device/sensor health (Parts 10, 12, 41, 44, 58–59)	HAL providers, ONNX Runtime, Docker	Publishes canonical events (Part 30) to Member 4's ingestion API
4 — Backend / Data / Fusion	FastAPI, PostgreSQL/PostGIS, event ingestion, evidence fusion, RoadTwin, Urban Memory, APIs (Parts 32–33, 35–36, 50–51)	FastAPI, PostGIS	Consumes Member 3's events; exposes API to Members 5 and 6
5 — Frontend / GIS	Maps, 20 GIS layers, RoadTwin visualization, maintenance dashboard, fleet, coverage (Part 46)	React, GIS mapping library	Consumes Member 4's API
6 — Integration / GenAI / Analytics	Copilot, RAG, authority workflows, analytics orchestration, integration testing, demo (Parts 47, 52–53, 62, 82)	LLM + tool-calling	Consumes Member 4's API; coordinates end-to-end integration and the demo

Cross-contribution is expected. Every hand-off between members must go through the canonical event schema (Part 30) or the API contract (Part 51), never an ad-hoc structure agreed informally between two people.

77. Repository

```
urbansense/
├── edge/           # HAL providers, edge agent entrypoint, Docker config for the edge container
├── models/        # trained/exported model artifacts (ONNX), model registry references
├── perception/    # detection / segmentation inference code (defects, infrastructure, vehicles,
VRU)
├── tracking/      # multi-object tracking
├── events/        # canonical event schema, serialization, validation (Part 30)
├── backend/       # FastAPI app, ingestion, API endpoints (Part 51)
├── database/      # PostgreSQL/PostGIS models, migrations (Part 50)
├── fusion/        # evidence-fusion engine, negative evidence (Parts 33-34)
├── roadtwin/      # RoadTwin state logic and lifecycle (Parts 35, 37)
├── gis/           # GIS layer definitions, map-serving endpoints (Part 46)
├── copilot/       # GenAI Copilot tool definitions (Part 52)
├── rag/           # SOP/policy retrieval (Part 53)
├── sensing/       # sensor health, coverage intelligence (Parts 41-42)
├── simulation/    # simulated fleet/GPS generation (Part 52 terminology)
├── datasets/      # dataset manifests/splits (Part 55) – not raw data itself
├── configs/       # per-environment config (laptop vs. edge execution provider, thresholds)
├── tests/         # unit/integration/e2e/acceptance tests (Parts 62, 64)
├── docker/        # Dockerfiles / compose for edge and backend
├── docs/          # this document and derived artifacts (PPT source, API docs)
└── scripts/       # dev tooling, data prep, demo setup
```

Refine subdirectories under each top-level folder as modules mature; the top-level boundaries stay stable so ownership (Part 76) maps cleanly onto directories.

78. API Contracts

The full, authoritative API contract is Part 51 — every endpoint, method, purpose and auth requirement a module owner needs is defined there. This section exists only to satisfy the document's navigation structure; do not duplicate Part 51 elsewhere.

79. Database Contracts

The full, authoritative database model is Part 50 — every entity and key relationship a module owner needs is defined there.

80. Event Contracts

The full, authoritative canonical event schema is Part 30 — every field, its type, and its required/optional status is defined there. Extensions to event_type (Part 30) must be logged in the decision log (Part 1.6) so all module owners stay in sync.

81. Definition of Done

Objective completion criteria for major components. A module is not 'done' on the basis of a demo looking right — it is done when every item below is checked.

Road-defect detection (Part 14) — DONE only when:

- Model runs on the selected dataset (Part 55).
- Inference pipeline works end-to-end on the edge HAL (Part 10).
- Bounding boxes/segmentation masks generated with a confidence score.
- Observation-quality score attached (Part 27).
- Evidence frame produced and stored.
- Canonical event generated (Part 30) with GPS attached.
- Event transmitted over the configured transport (Part 31).
- Backend stores the event and links it to a road segment (Part 32, 50).
- GIS displays it on the Road Health layer (Part 46).
- Unit + integration tests pass (Part 62).

Infrastructure intelligence (Part 15) — DONE only when:

- Expected-asset inventory (Part 15.3) is populated for the demo route.
- Expected-vs-observed comparison runs against live observations.
- Negative-evidence gate (Part 34) is applied before any 'missing' conclusion.
- Present/damaged/absent/uncertain states are all distinguishable in the API/GIS.
- Multi-pass corroboration is required before 'absent' is confirmed.
- Tests cover both a present-asset and a genuinely-missing-asset case (Scenario 3, Part 64).

Evidence fusion + RoadTwin (Parts 33, 35, 37) — DONE only when:

- Independent/duplicate/correlated observation classification (Part 33.1) is implemented, not just a raw count.
- All seven lifecycle states are reachable and testable (Scenario 1, Part 64).
- Negative evidence (Part 34) gates every absence-based transition.

- RoadTwin exposes the full component breakdown (Part 35.1), not a single opaque score.
- Freshness/decay is tracked and queryable (Part 35.2).
- API (Part 51) and GIS (Part 46) both read from RoadTwin, not from raw events directly.

Maintenance + repair verification (Parts 38-40) — DONE only when:

- Priority scoring runs with documented, PROPOSED weights (Part 38).
- Repair-report workflow is available to an authority user (Part 47).
- Verification-pending monitoring actually raises re-observation priority (Part 45).
- Verified-repaired and reappeared transitions are both demonstrable (Scenario 1 and 13, Part 64).

GenAI Copilot (Part 52) — DONE only when:

- At least the five example queries (Part 52) work via tool calls, not hard-coded text.
- Every answer is traceable to an actual backend query result (Scenario 15, Part 64).
- RBAC scoping is enforced per query (Part 49).
- The Copilot cannot be shown to fabricate a road condition not present in the backend (adversarial test in Part 62 security tests).

GIS (Part 46) — DONE only when:

- All 20 layers render against test data, even if some contain only demo-scale content.
- ANPR-restricted layer correctly hides content from an unauthorized role.
- Historical RoadTwin layer supports at least a basic time-scrub for the demo route.

82. Demo Architecture

Primary narrative (do not broaden scope beyond this if demo time is limited):

```

BUS-001
↓
Pothole detected
↓
Evidence generated
↓
GIS location
↓
Central platform
↓
BUS-002 observes same location
↓
Evidence fusion (independence-checked, Part 33.1)
↓
RoadTwin = CONFIRMED
↓
Maintenance action
↓
Repair reported
↓
Future bus passes
↓
Confident absence (Part 34)
↓
VERIFIED_REPAIRED

```

Then, time permitting, show as secondary beats: a bottleneck/route-delay view, one incident with ANPR evidence and its full evidence graph (Part 22.1), the sensing-coverage map (Part 42), and a short Copilot query

(Scenario 15). Do not attempt to demo every module in Part 73 if it risks the core lifecycle story running long or breaking — the core loop is the single most important thing judges must see work end to end.

83. SIH PPT Source of Truth

Slide-by-slide source material — this is what the PPT is built from, not the PPT itself.

Slide	Objective	Key message	Bullets	Diagram/visual	Source
1. Title	Introduce the project and team	UrbanSense AI — PS 26124, team name	N/A	Title art / bus + city skyline motif	Part 2
2. Problem	Establish the gap	Fixed CCTV, manual inspection, complaints are slow and incomplete (Part 5)	Existing-approach limitations	Simple before/after diagram	Part 5
3. Complete PS scope	Show we address every requirement	31 traced requirements across 4 groups (Part 4)	Group summary	Traceability-matrix excerpt	Part 3-4
4. Proposed solution	Introduce UrbanSense	Buses as mobile sensors + evidence-driven fusion (Part 6.1)	3-5 bullets	Master architecture diagram (Part 8)	Part 6, 8
5. Why different	Core differentiator	Evidence, not one-shot detection; lifecycle-aware RoadTwin (Part 6.4)	Positioning statement	Lifecycle diagram	Part 6.4, 37
6. Architecture	Show the full pipeline	Layer-by-layer flow (Part 8)	Layer list	Master architecture diagram	Part 8
7. Edge AI	HAL + adaptive fast/slow-path processing	Provider abstraction, laptop-to-Jetson portability (Part 10), adaptive compute (Part 44)	HAL interfaces	HAL + fast/slow-path diagram	Part 10, 12, 44
8. Road & infrastructure intelligence	Defect + missing-infrastructure detection	Detection + expected-asset reasoning (Parts 14-15)	Key capabilities	Sample defect/infrastructure crops	Part 14-15
9. Traffic & OD intelligence	Broader vehicle-based analytics	Density, bottleneck, route delay, OD (Parts 16-20)	Key capabilities	Sample dashboard mock	Part 16-20
10. Safety & incident intelligence	VRU risk + hit-and-run/ANPR	Risk scoring, evidence graph, ANPR confidences (Parts 21-26)	Workflow steps	Incident-evidence-graph diagram	Part 21-26
11. Evidence fusion / RoadTwin	The core innovation	Negative evidence + lifecycle state machine (Parts 33-37)	State machine summary	State-machine diagram	Part 37
12. GIS dashboard	Authority-facing view	20-layer map (Part 46)	Layer list (condensed)	Dashboard screenshot/mock	Part 46
13. Authority workflows	From data to decision	Four authority action paths (Part 47)	Workflow diagram	Workflow diagram	Part 47
14. GenAI Copilot	Constrained assistant	What it can/cannot do (Part 52)	CAN/CANNOT list	Copilot query example	Part 52
15. Innovation portfolio	Recap differentiators	Lifecycle, negative evidence, cooperative sensing, adaptive edge (Parts 33.1, 34, 37, 44)	Novelty matrix excerpt (Part 71.3)	N/A	Part 71
16. Feasibility	De-risk for judges	Laptop-first HAL, honest simulation approach (Part 52 terms)	MVP scope (Part 72)	Roadmap snippet	Part 10, 72, 75
17. Data / datasets	Show grounding	Dataset matrix (Part 55)	Key datasets used	Dataset table (condensed)	Part 55
18. Implementation	Team + plan	Six-member division + phased roadmap (Parts 75-76)	Phase highlights	Roadmap timeline	Part 75-76

19. Expected impact	Value to authority	Closed-loop maintenance, safer roads, verified repairs	3 impact bullets	N/A	Part 39, 47
20. Research potential	Academic angle	Lifecycle/negative-evidence research question (Part 65)	1-sentence research question	N/A	Part 65-71

This is source material for the PPT — the final slide deck should still be written and designed separately, not generated verbatim from this table.

84. Judge Questions

Category	Question	Concise answer	Supporting part
Problem	Why not just use more fixed CCTV?	Fixed cameras only cover a few points; buses already traverse nearly every road daily.	Part 5
Scope	Do you cover the whole problem statement, or just potholes?	Every PS-26124 capability is traced in Part 4's matrix — defects, infrastructure, traffic, VRU, incidents, ANPR, GIS, and analytics are all in scope.	Part 3-4, 73
AI	What models are you using?	A detection/tracking/OCR pipeline fine-tuned for Indian road conditions; exact variants are being benchmarked.	Parts 14-16, 26, 56 — OPEN DECISION on exact variant
Edge AI	Do you need special hardware?	No — it runs on a laptop today via a hardware abstraction layer, and the same container moves to Jetson-class hardware later with only a config change.	Part 10
Efficiency	Doesn't running many AI tasks kill battery/compute?	We use an adaptive fast-path/slow-path architecture — heavy models only run when uncertainty or risk justifies it.	Part 12, 44
Architecture	What happens if the network drops?	The edge agent queues events locally and retries; nothing is lost, just delayed.	Part 59-60
Datasets	What data are you training on?	A mix of public Indian-driving datasets (IDD, BDD100K, Road Damage Dataset) plus custom-collected footage.	Part 55
Accuracy	What accuracy do you get?	We report only measured numbers from our own benchmarking, not assumed figures.	Part 63, 85
Evidence Fusion	Why not just trust each detection directly?	A single frame can be a false positive/negative; we require corroboration across time and buses — and we distinguish genuinely independent corroboration from duplicate or correlated reports.	Part 33, 33.1
RoadTwin	What is RoadTwin exactly?	A persistent, per-road-segment state record — not a one-off detection, but an evolving picture of that	Part 35

		segment's condition across all its dimensions.	
Missing infrastructure	How do you detect something that ISN'T there?	We compare against an expected-asset inventory from GIS, and only declare 'missing' after multiple high-visibility passes fail to find it.	Part 15, 34
ANPR	How do you extract plates?	Only on incident trigger, using multi-frame cropping and OCR, with plate detection and OCR confidence tracked separately.	Part 26
Privacy	What about passenger privacy?	Cabin cameras are scoped strictly to passenger-safety use cases and excluded from road/traffic/ANPR pipelines.	Part 49
Trust/Auditability	Why should we trust a 'confirmed' pothole?	Every confirmed state can be traced back to the exact contributing observations, buses, and timestamps that produced it.	Part 42 (via Parts 22.1, 33)
Scalability	Does this scale beyond one bus?	Yes — the event schema and database model are fleet-scale by design; we're demonstrating with a simulated subset.	Parts 30, 50, 75
Cost	Is this affordable for a city to deploy?	A tiered fleet (mostly low-cost edge nodes, a small share of higher-compute nodes) keeps per-bus cost realistic; exact figures require a procurement-level costing exercise.	Part 59 note — PROPOSED, not finalized
Deployment	How would this actually roll out?	Laptop prototype → Jetson-class/industrial-PC pilot fleet → full fleet, with no application-logic rewrite required at each step.	Part 10, 75
GenAI	What does the AI chatbot actually do?	It answers questions and summarizes evidence using our backend's real data — it cannot invent facts or bypass the deterministic logic.	Part 52
Research novelty	What's actually new here?	Not the individual components — the lifecycle-aware, quality-weighted, negative-evidence-driven fusion model that turns detections into verified, trackable road-state history.	Parts 65, 70-71
Competition	How is this different from existing pothole-detection apps?	Most stop at single-shot detection; we model the full lifecycle including repair verification and recurrence, using independence-checked multi-bus corroboration.	Part 39-40

85. What We Must Not Claim

- We do not currently have physical deployment on government buses.

- We do not claim access to proprietary bus CCTV.
- We do not claim live fleet telemetry from any operator unless it is actually obtained.
- We do not claim experimental accuracy, FPS, or bandwidth-savings figures before they are actually measured (Part 63).
- We do not claim research novelty without literature validation (Part 71).
- We do not claim that GenAI performs the core computer vision — it only interprets and summarizes (Part 52).
- We do not claim that a single non-detection proves a defect is absent — absence requires confident, quality-gated evidence (Part 34).
- We do not claim that combining known technologies automatically constitutes novelty (Part 71.1).
- We do not claim production bus GPS or IMU data — the prototype uses simulated GPS/IMU.
- We do not present a specific maintenance-priority formula, bandwidth figure, or cost figure as finalized — these are PROPOSED pending validation (Parts 38, 59).
- We do not claim a full origin-destination matrix — only route/corridor-level approximations from fleet trajectories (Part 20).
- We do not claim ANPR functions as, or will become, a general-purpose surveillance/search capability (Part 49.1).

85.1 What is real vs. simulated

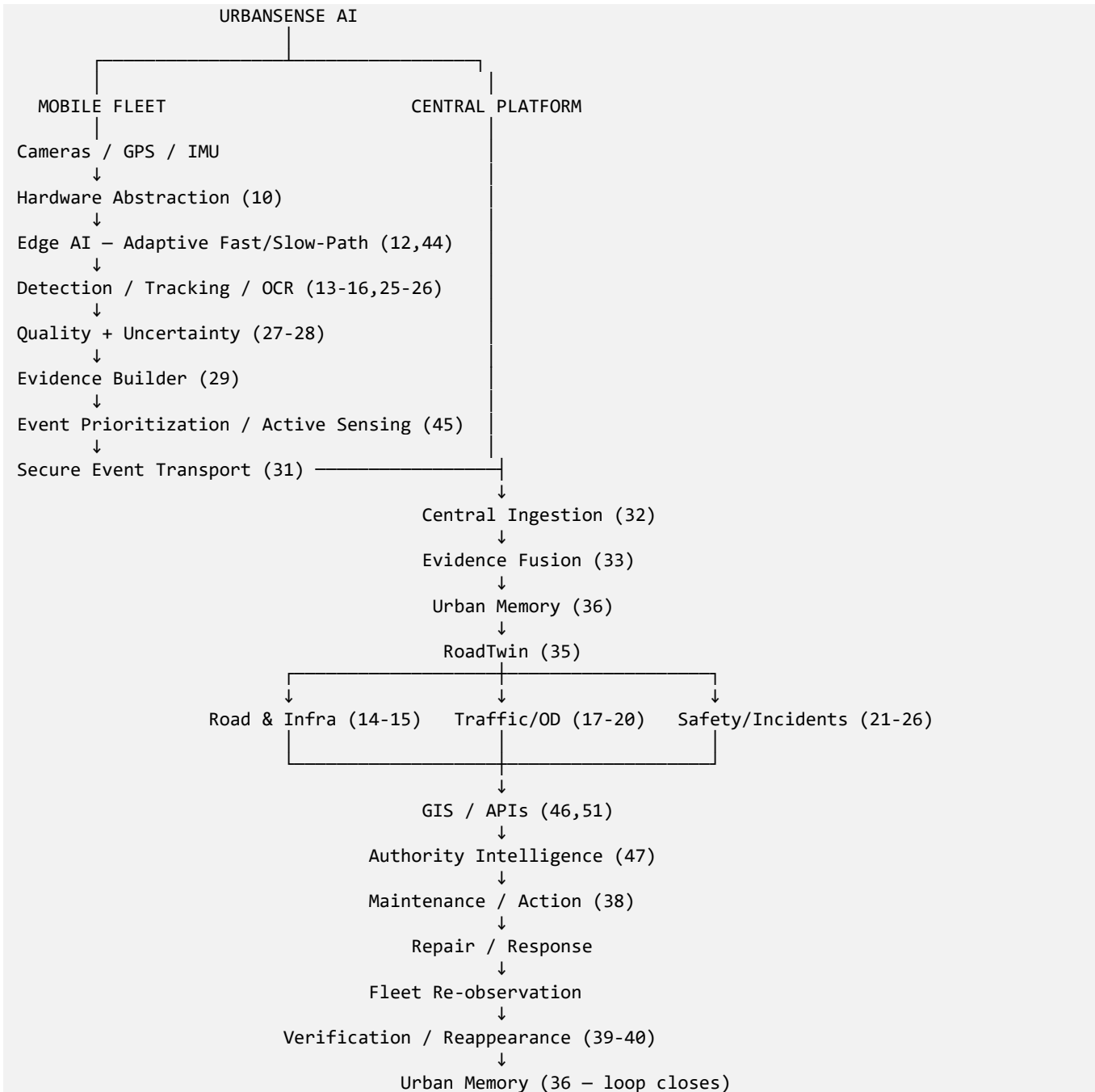
Component	Prototype	Production
Bus identity	Simulated	Real bus
Camera	Recorded video (FileCameraProvider)	Bus camera / RTSP or MIPI feed
GPS	Simulated/replayed (SimulatedGNSSProvider)	Real GNSS
IMU	Simulated/optional	Real IMU
Fleet scale	Simulated (few buses)	Actual fleet
Road network	Real geography	Real geography
AI models	Real inference	Real inference
GIS	Real geographic context	Real geographic context
Backend	Real (FastAPI/PostGIS)	Real
RoadTwin	Real (persisted state logic)	Real

86. Open Decisions

- Exact model variant(s) per CV task (defect, infrastructure, vehicle, VRU detector; tracker; OCR) — Parts 14-16, 21, 26, 56-57.
- Exact datasets/footage for the Hyderabad demo route — Part 55.
- Exact GPS/IMU simulation method and trajectory source — Part 10, 52.
- Exact mathematical formulation of the evidence-fusion engine (Bayesian updating vs. Dempster-Shafer vs. a simpler MVP heuristic) — Parts 33, 69.
- Exact confidence/quality/absence thresholds for state transitions — Part 37.
- Exact number of independent corroborating observations (N) required for CANDIDATE→CONFIRMED — Part 37.
- Exact database schema (column-level) — Part 50.
- Exact object storage choice for evidence media — Part 32.
- Exact LLM provider/hosting for the Copilot — Part 52.
- Exact vector database (if RAG is implemented) — Part 53.

- Exact GIS mapping library (MapLibre vs. Leaflet) and which of the 20 layers ship for the demo vs. are stubbed — Part 46.
- Exact deployment hardware for the production pitch (Jetson Orin Nano/NX vs. an industrial x86 mini-PC) — Part 10.
- Exact maintenance-priority weighting formula — Part 38.
- Exact freshness/decay function for RoadTwin state — Part 35.2.
- Which theoretical anchor (POMDP/factor-graph vs. information-theoretic active sensing) the research paper builds its proofs around — Part 69.
- Whether Evidential Deep Learning / conformal prediction is attempted for the SIH build or deferred entirely to the research track — Part 28.
- Exact scope of Active Sensing (Part 45) implemented for the Full PS build vs. left as the research-tier formulation only.

87. Final Master Blueprint



Two halves — a mobile fleet that senses and a central platform that remembers — connected by a single secure, prioritized event stream. The mobile half turns raw sensor data into quality-scored evidence, allocating its own compute adaptively; the central half turns that evidence into a persistent, corroborated, continuously verified model of the city's full PS-26124 scope, closing the loop from detection through repair back to verification and, where it recurs, back into the same lifecycle. Every part of this document (1-86) exists to make one of these boxes concrete enough to trace to a PS requirement, build, test, present, or publish.

88. Master Project Snapshot

Problem — Public-transport buses cross nearly every road daily but their cameras aren't used for sensing; authorities rely on slow, incomplete fixed CCTV, manual inspection, and complaints (PS 26124, Part 3).

Complete solution — UrbanSense AI turns the bus fleet into mobile sensors covering every PS-26124 capability — road defects, missing/damaged infrastructure, vehicle/traffic analytics, vulnerable-pedestrian risk, and driving incidents with ANPR — and a central engine fuses these observations across buses and time into a persistent, auditable road state (Part 4 traceability matrix).

Core architecture — Fleet (HAL → Adaptive Edge AI → Evidence) → Secure Transport → Central Platform (Ingestion → Evidence Fusion → Urban Memory → RoadTwin → Analytics/GIS/Copilot) → Authority Action → Verification (Parts 8, 87).

All PS capabilities — 31 requirements across road/infrastructure, vehicle/traffic, safety/incident, and platform/analytics groups, each traced to a specific module (Part 4).

Main innovation — Lifecycle-aware, quality-weighted, negative-evidence-driven evidence fusion — a defect is SUSPECTED, CONFIRMED, repaired, and independently VERIFIED or found REAPPEARED, with a non-detection treated as evidence only once visibility is accounted for (Parts 33-34, 37).

Research contribution — Isolated lifecycle-aware, uncertainty-driven evidence fusion for road-state estimation and repair verification, benchmarked against classical truth-discovery and temporal-fusion baselines (Parts 65-70).

MVP — Recorded video → edge pothole detection → GPS association → event → MQTT → FastAPI → PostGIS → basic fusion → RoadTwin → GIS dashboard — the first vertical slice, not a scope ceiling (Part 72).

Full implementation scope — Every module in Parts 14-54, staged across a 26-phase roadmap (Part 75), delivering complete PS-26124 coverage (Part 73).

Team structure — Six members: Computer Vision, Tracking/Motion/Traffic, Edge/Systems, Backend/Data/Fusion, Frontend/GIS, Integration/GenAI/Analytics (Part 76).

Demo story — BUS-001 detects a pothole → BUS-002 independently corroborates it → RoadTwin CONFIRMED → maintenance → repair reported → future bus passes confirm absence → VERIFIED_REPAIRED, with bottleneck, incident/ANPR, coverage-map and Copilot beats time permitting (Part 82).

Deployment path — Laptop prototype → tiered edge-hardware fleet (mostly low-cost nodes, a smaller share of higher-compute survey nodes) → full-fleet rollout, with no application-logic rewrite at each step (Parts 10, 75).

What we must not claim — No physical government deployment, no proprietary CCTV/live fleet access, no unmeasured accuracy/bandwidth/cost figures, no unvalidated research-novelty claims, and no GenAI-invented facts (Part 85).